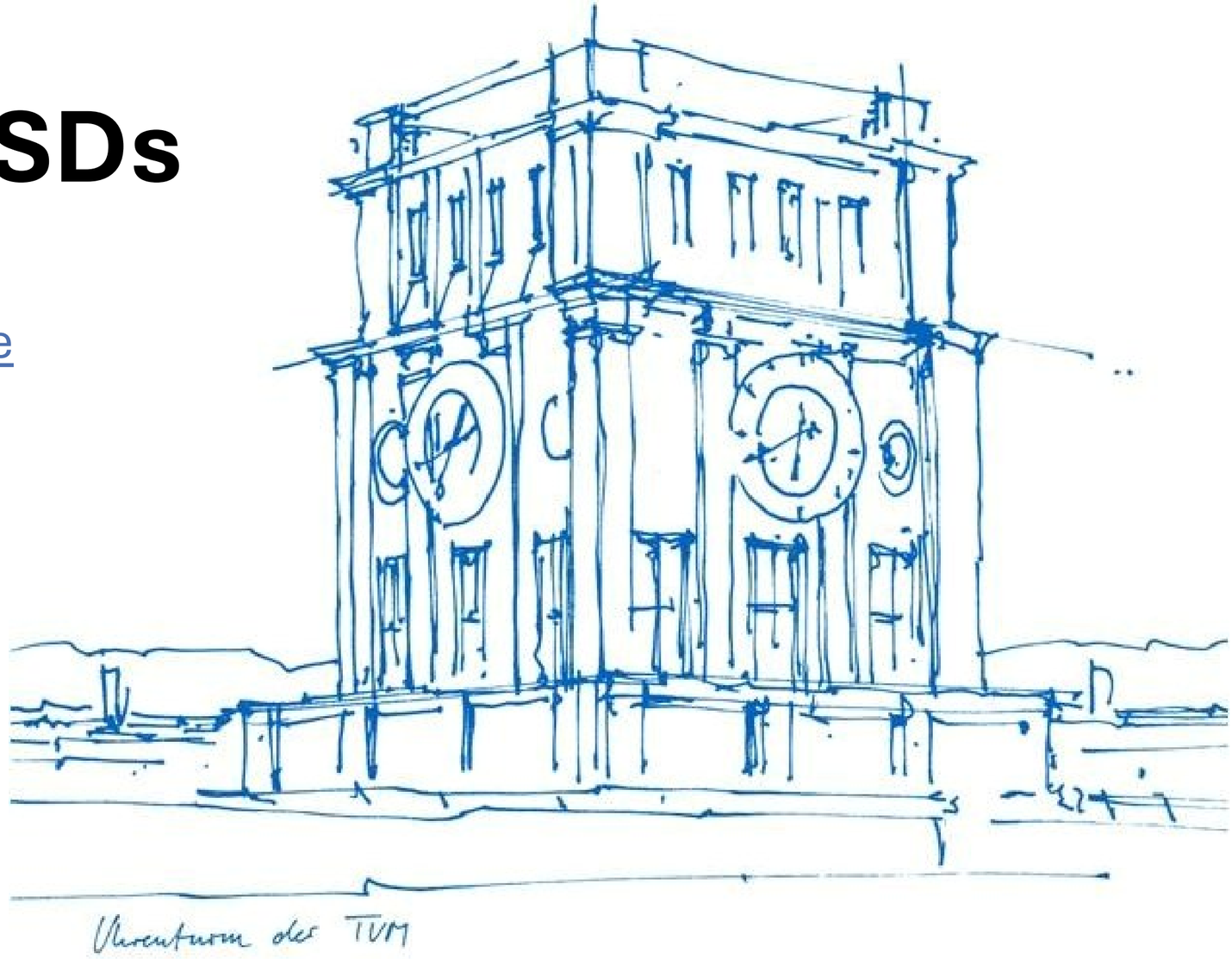
How to Write to SSDs

Bohyun (Lia) Lee bohyun.lee@tum.de

with **Viktor Leis** and **Tobias Ziegler**

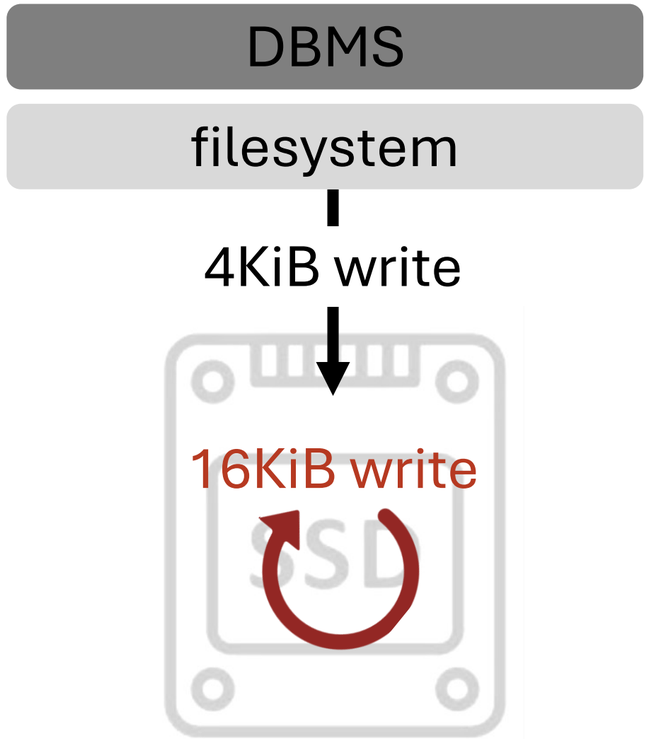


How Should DBMSs Write to SSDs?

Why Should DBMSs Care About How They Write to SSDs?

SSD writes are different from disk writes

- SSD writes are special due to internal *write amplification*
- *e.g.*, 4KiB write request can be amplified to 16KiB



Implications of SSD WAF

- High SSD WAF leads to
  - lower SSD throughput → lower DBMS throughput

Implications of SSD WAF

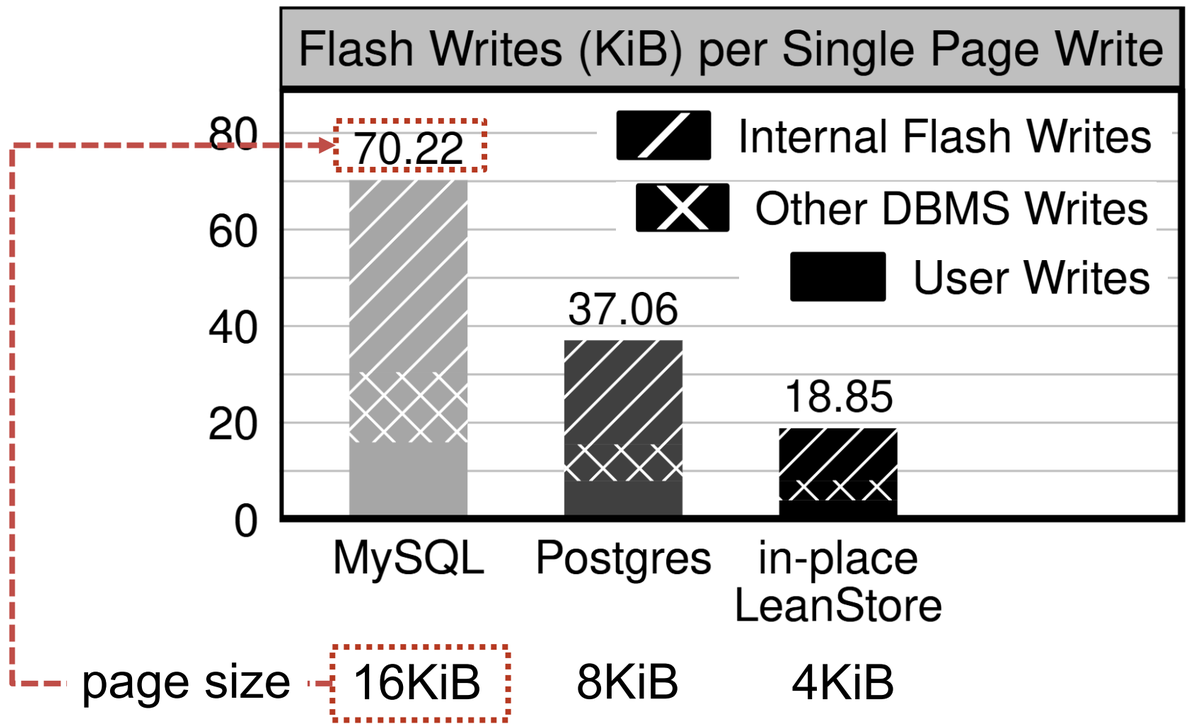
- High SSD WAF leads to
  - lower SSD throughput → lower DBMS throughput
  - reduced SSD lifespan
    E.g., 400MB/s writes break an SSD (5 DWPD) in 2 months

Implications of SSD WAF

- High SSD WAF leads to
  - lower SSD throughput → lower DBMS throughput
  - reduced SSD lifespan
    E.g., 400MB/s writes break an SSD (5 DWPD) in 2 months
- ***In general, fewer writes, the better!***

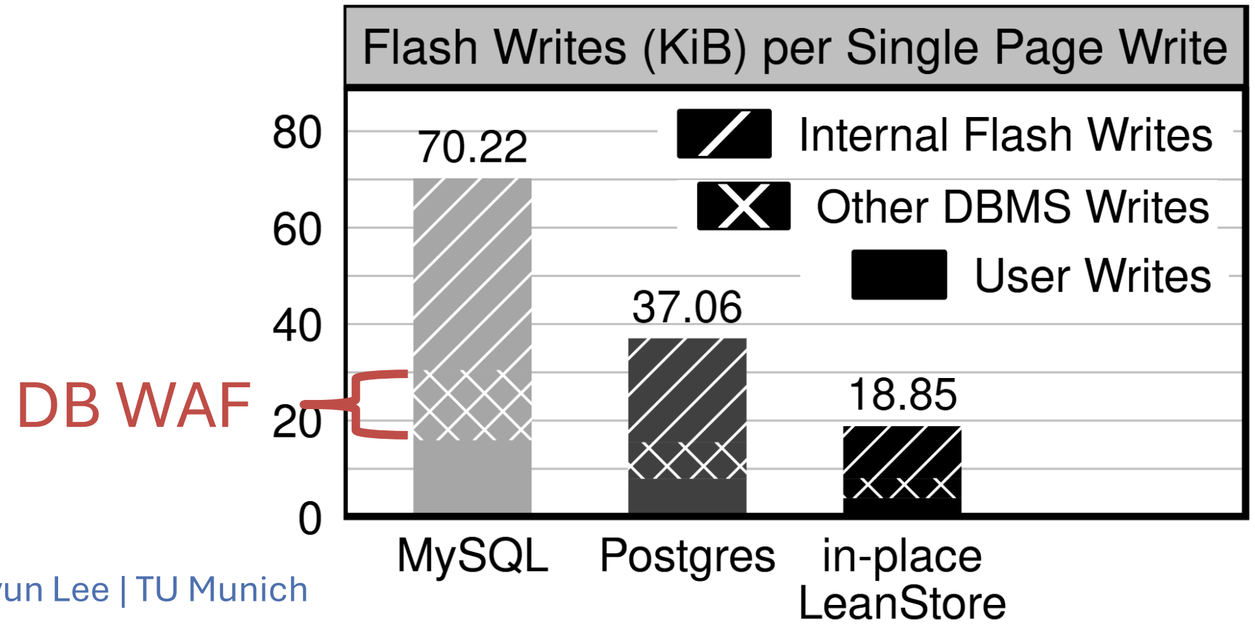
DBMSs suffer from write amplification

- User writes are amplified by 4.3-4.7x in three DBMSs



Two culprits behind write amplification

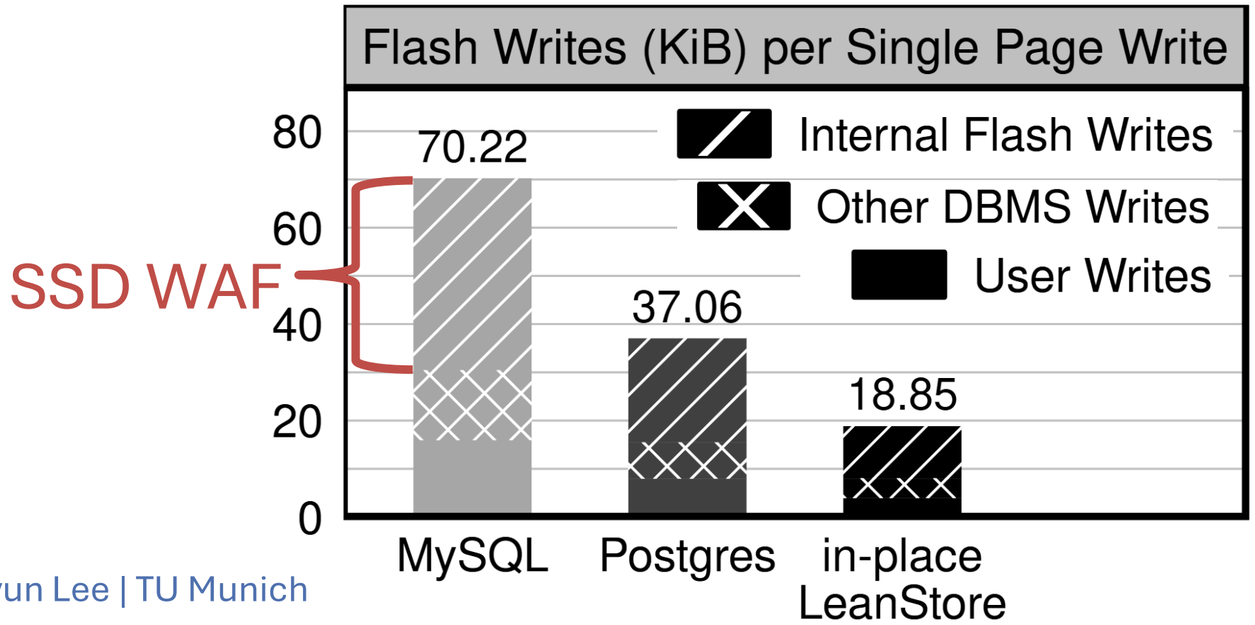
- Culprit 1: DBMSs themselves write more
  (i.e., torn page write protection *doubles* the writes)



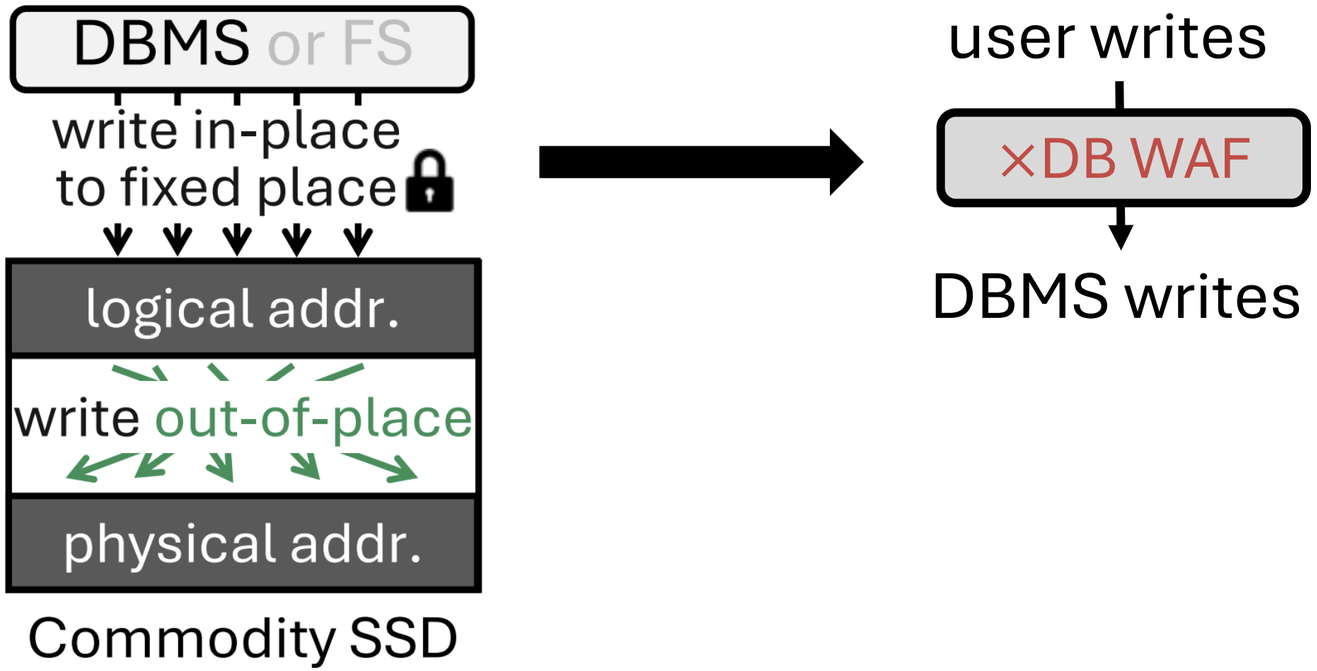
Two culprits behind write amplification

- Culprit 1: DBMSs themselves write more
  (i.e., torn page write protection *doubles* the writes)

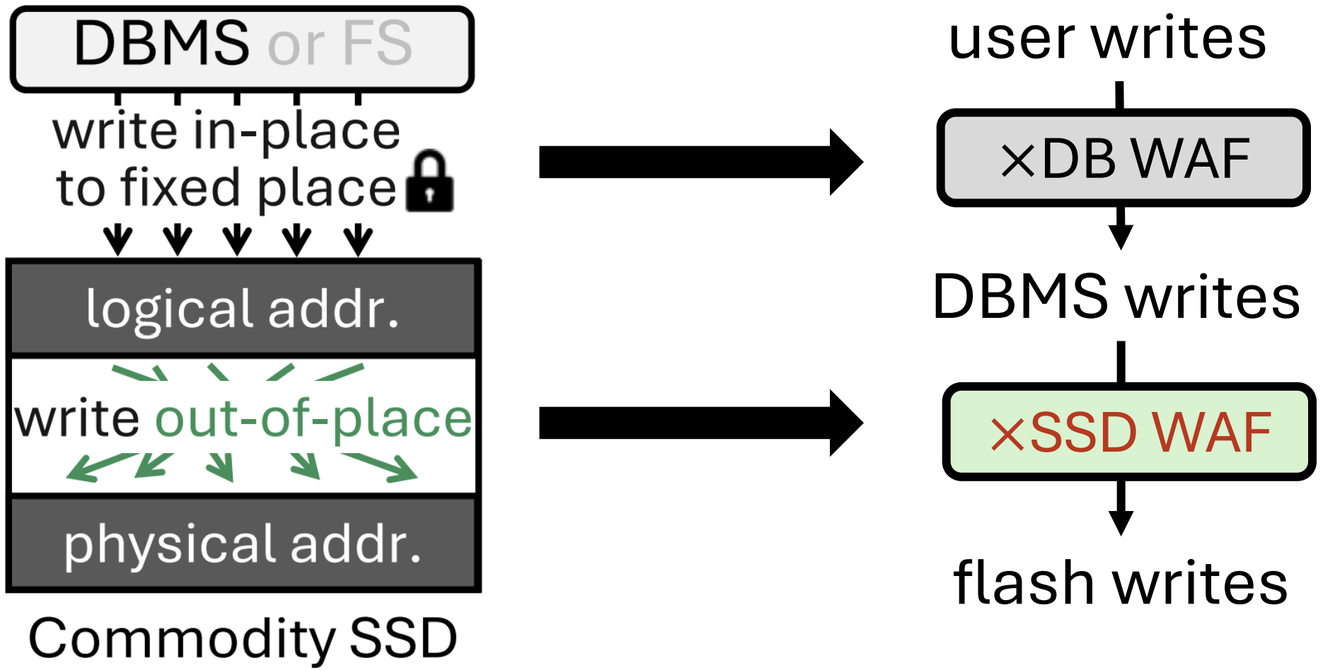
- Culprit 2: SSDs internally amplify DBMS writes further



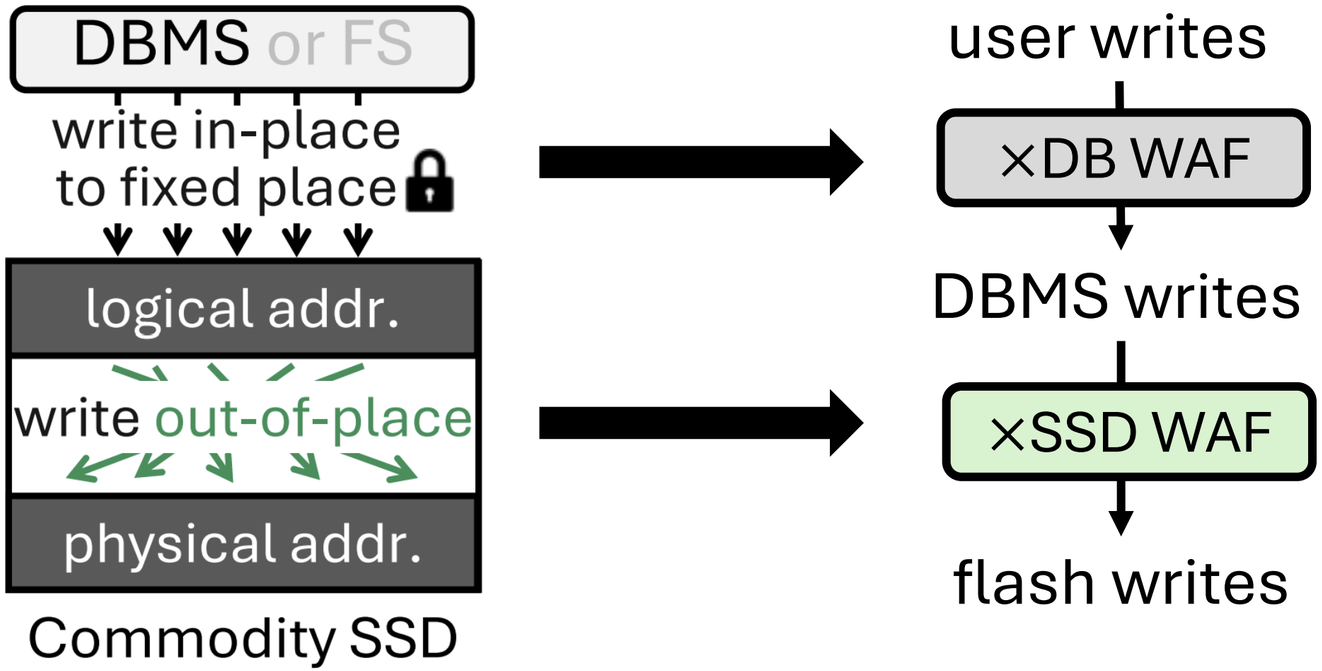
Total WAF


DBMS or FS
write in-place
to fixed place
logical addr.
write out-of-place
physical addr.
Commodity SSD
user writes
×DB WAF
DBMS writes

Total WAF

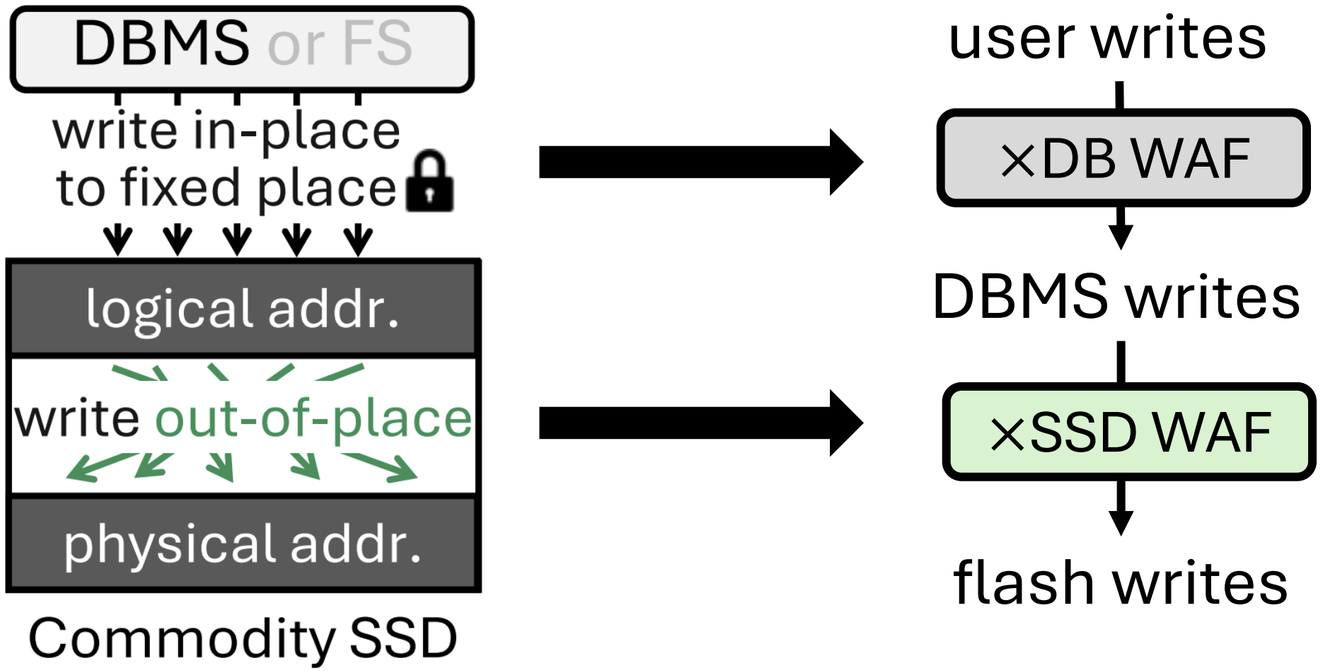

DBMS or FS
write in-place
to fixed place
logical addr.
write out-of-place
physical addr.
Commodity SSD
user writes
×DB WAF
DBMS writes
×SSD WAF
flash writes

Total WAF



total WAF* = *DB WAF* × *SSD WAF

Total WAF

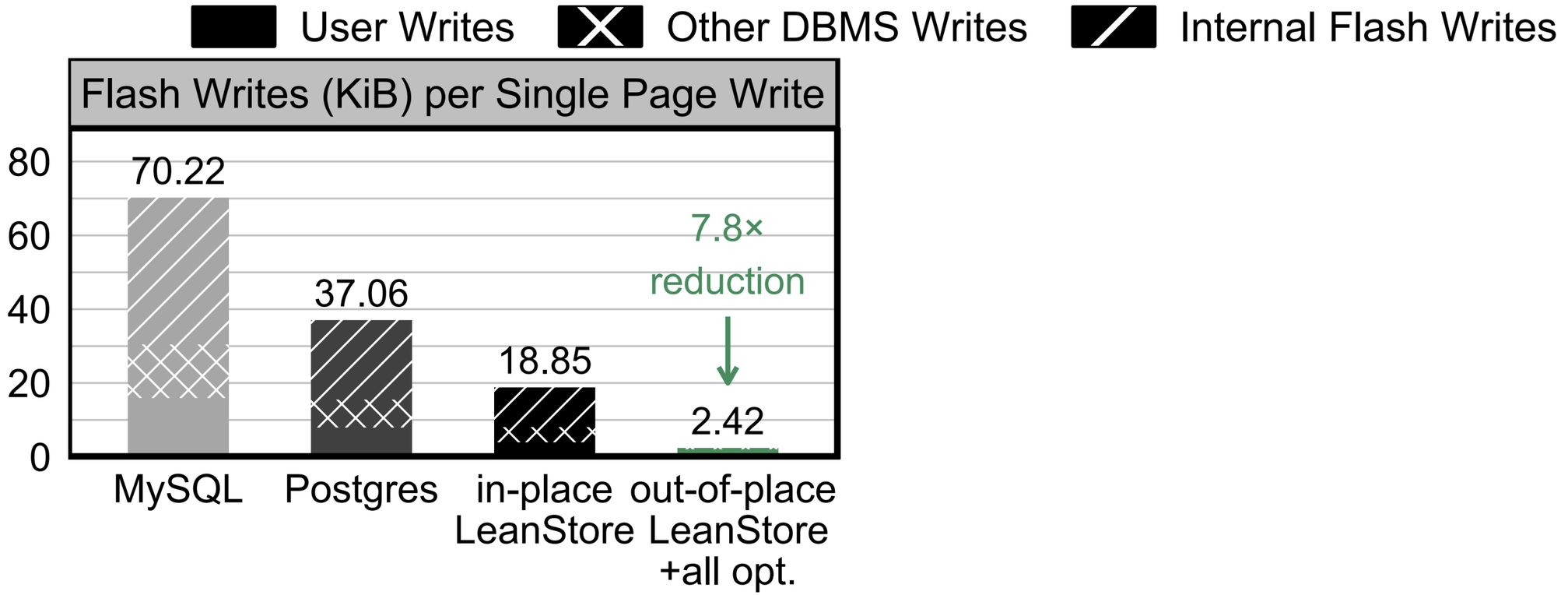


total WAF* = *DB WAF* × *SSD WAF

Database system should address it

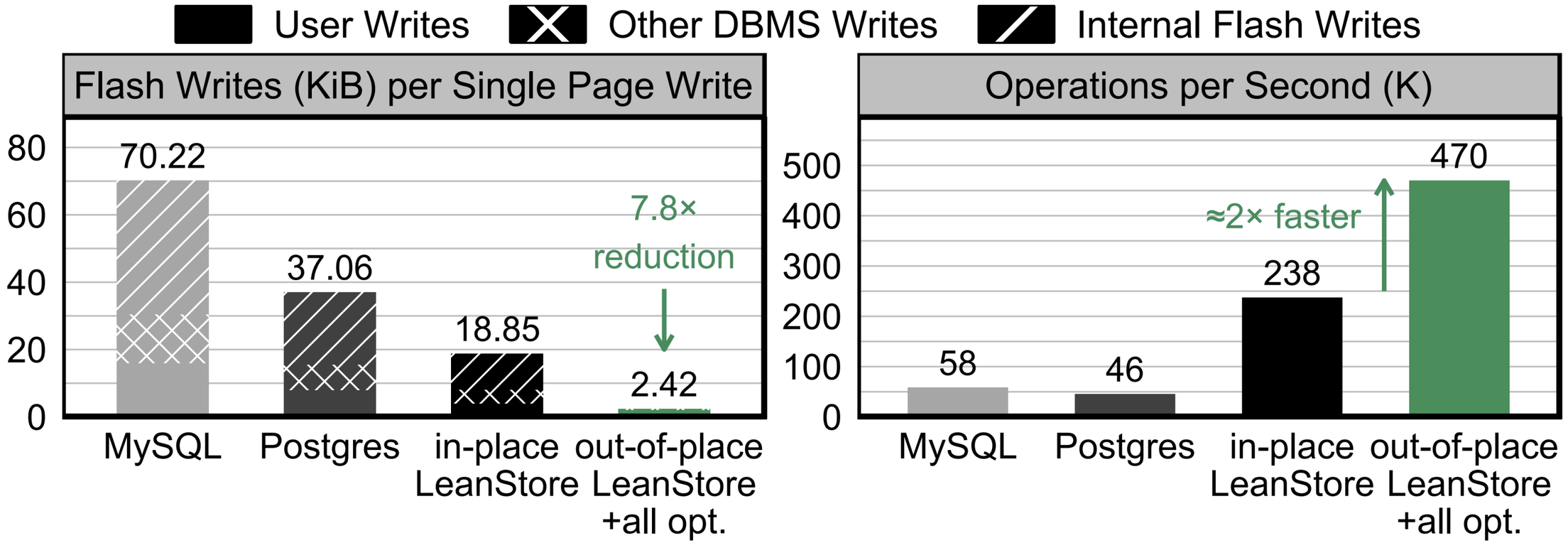
Result once you address total WAF

- Can achieve 7.8x flash write reduction



Gain once you address total WAF

- Can achieve 7.8x flash write reduction and 2x throughput



How to address total WAF?

Proposed optimizations

in-place write

↓

switch to *out-of-place write*

↓

+ compression and page packing

↓

+ deathtime-based placement & GC

↓

+ aligning DB SSD GC unit size

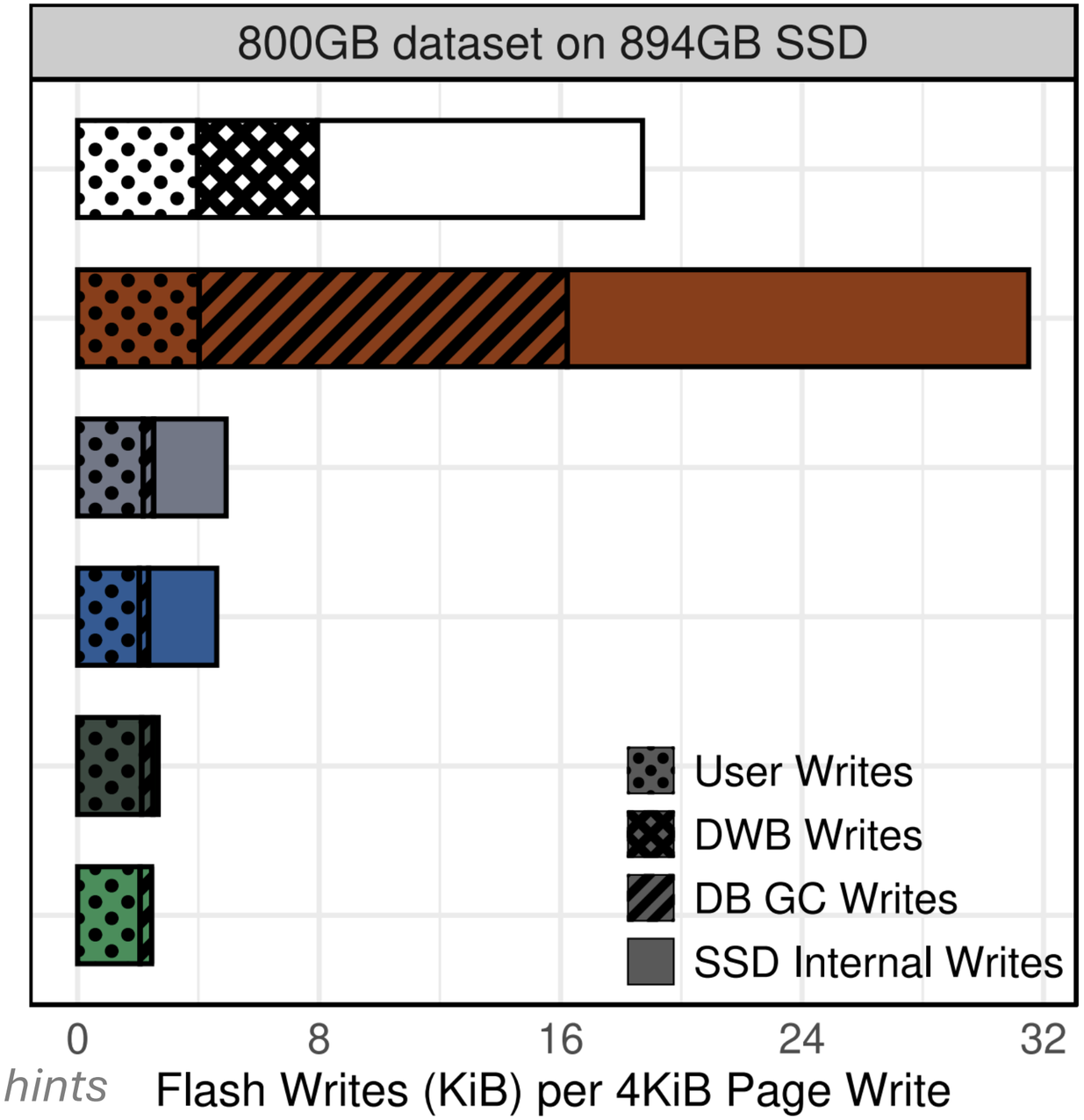
↓

+ *SSD WAF = 1*

ZNS
zone cmd

standard SSD
NoWA pattern

FDP
placement hints



To be on the same page

- DBMS is running an out-of-memory OLTP workload, I/O bound
- Writes happen directly to an SSD (no filesystem)
- Eviction or checkpoint writes have been triggered (WAL is located elsewhere)
- Dirty page writes are issued in a batch (e.g., flush 64 pages)

Proposed optimizations

in-place write

↓

switch to *out-of-place write*

↓

+ compression and page packing

↓

+ deathtime-based placement & GC

↓

+ aligning DB SSD GC unit size

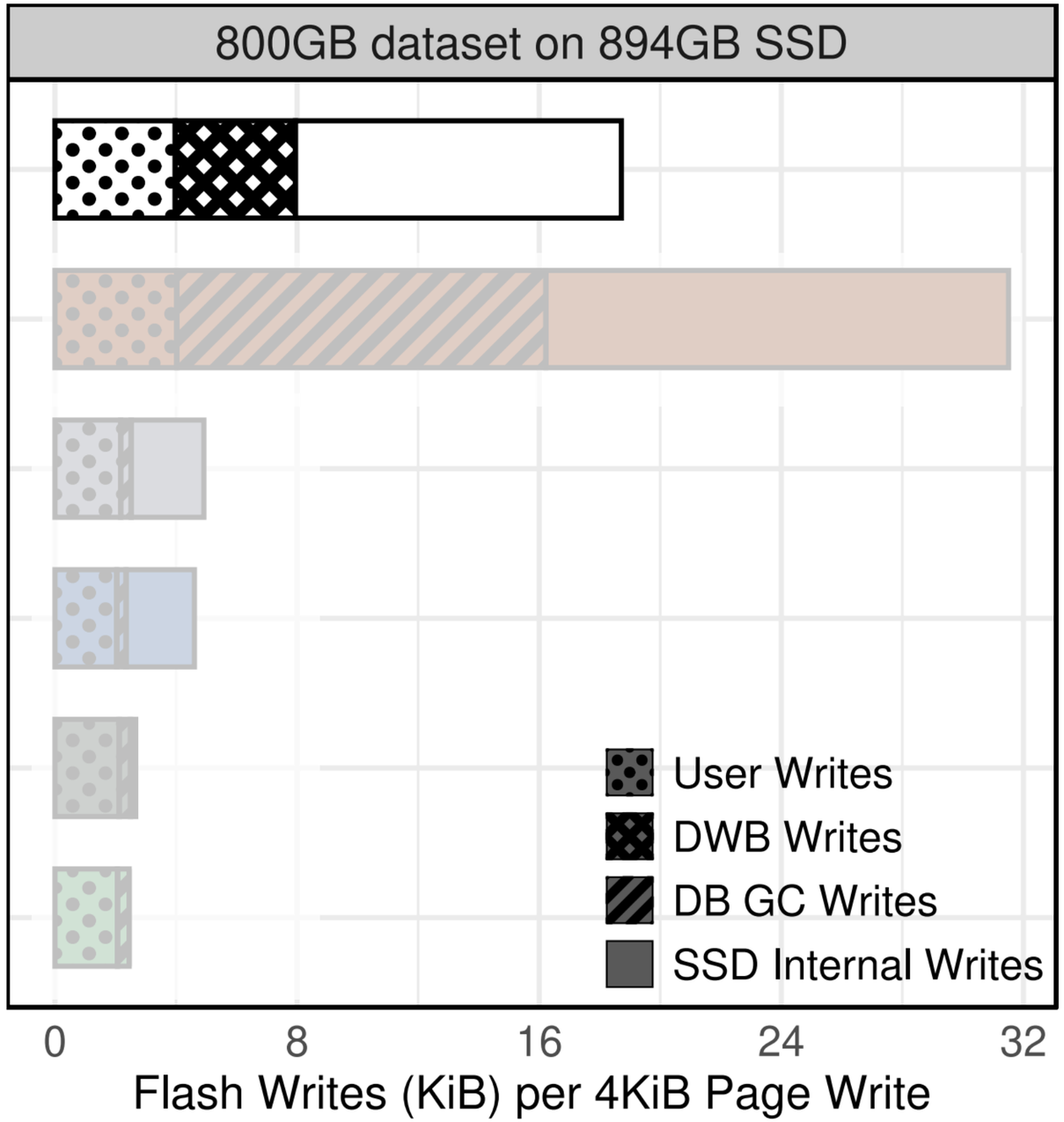
↓

+ *SSD WAF = 1*

ZNS
zone cmd

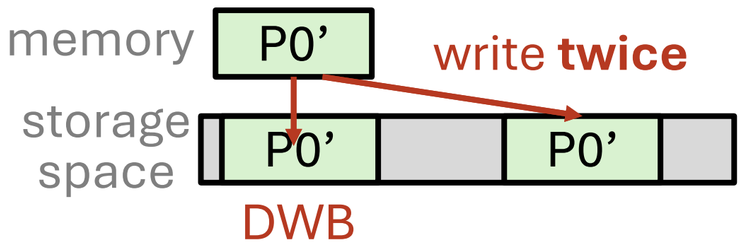
standard SSD
NoWA pattern

FDP
placement h

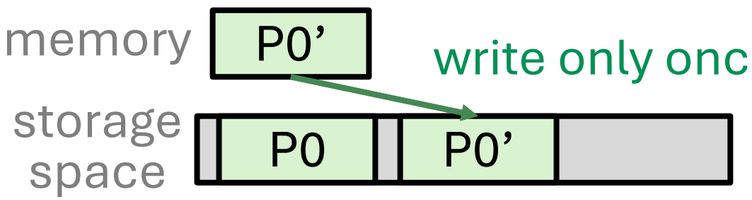


Out-of-place write is necessary because

- Out-of-place write minimizes **doublewrite buffering writes (DWB)** to protect from torn page writes
  - out-of-place writes can nearly **halve** the logical write



in-place write



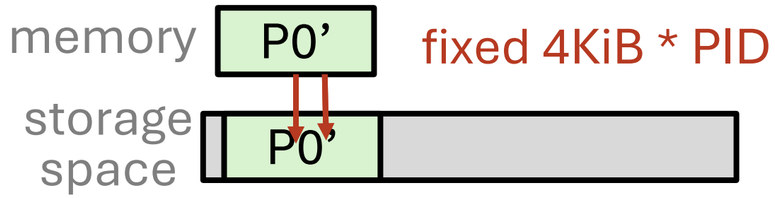
out-of-place write

Out-of-place write is necessary because

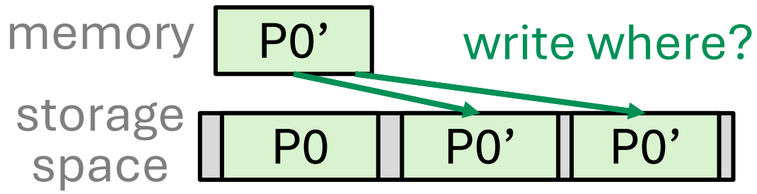
- Out-of-place write offers ***flexibility*** to tailor the write pattern according to system's needs in terms of

Out-of-place write is necessary because

- Out-of-place write offers ***flexibility*** to tailor the write pattern according to system's needs in terms of
  - **placement** (place P0 LBA [8-15] or…?)



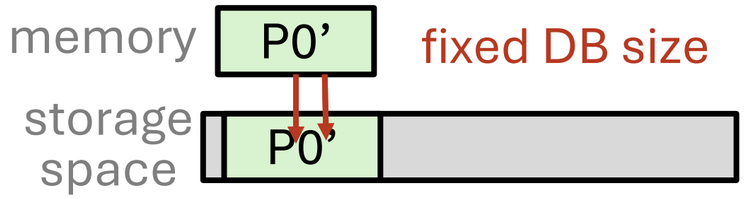
in-place write



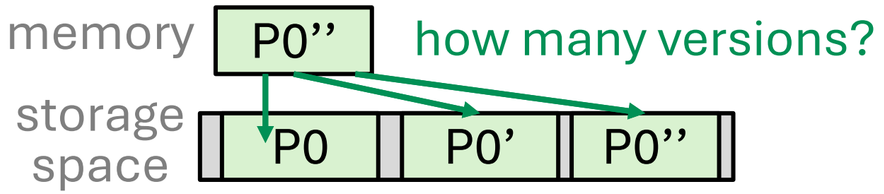
out-of-place write

Out-of-place write is necessary because

- Out-of-place write offers ***flexibility*** to tailor the write pattern according to system's needs in terms of
  - placement (place P0 LBA [8-15] or…?)
  - **storage size** (how big can the database grow?)



in-place write



out-of-place write

Proposed optimizations

in-place write

↓

switch to *out-of-place write*

↓

+ compression and page packing

↓

+ deathtime-based placement & GC

↓

+ aligning DB SSD GC unit size

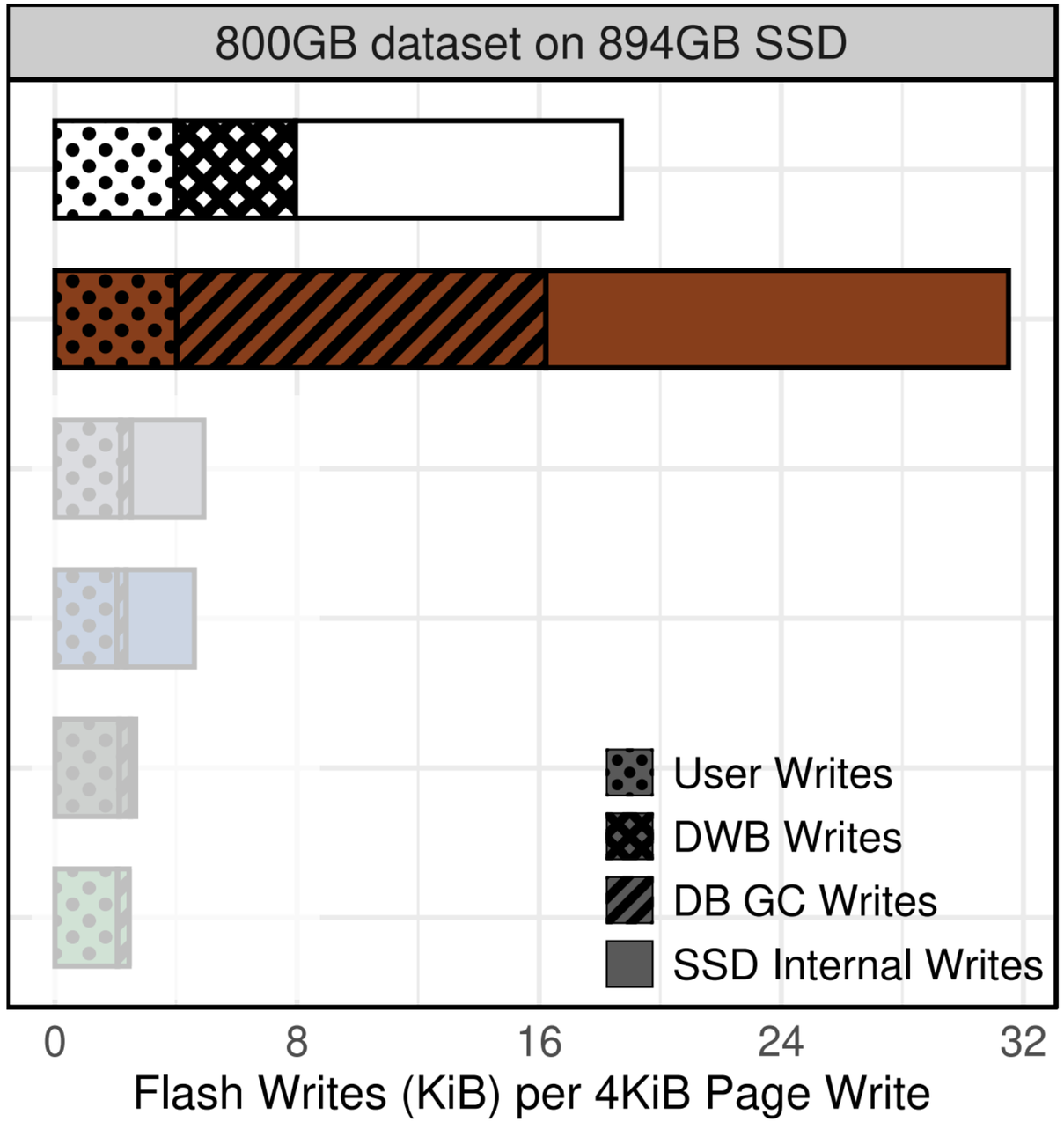
↓

+ *SSD WAF = 1*

ZNS
zone cmd

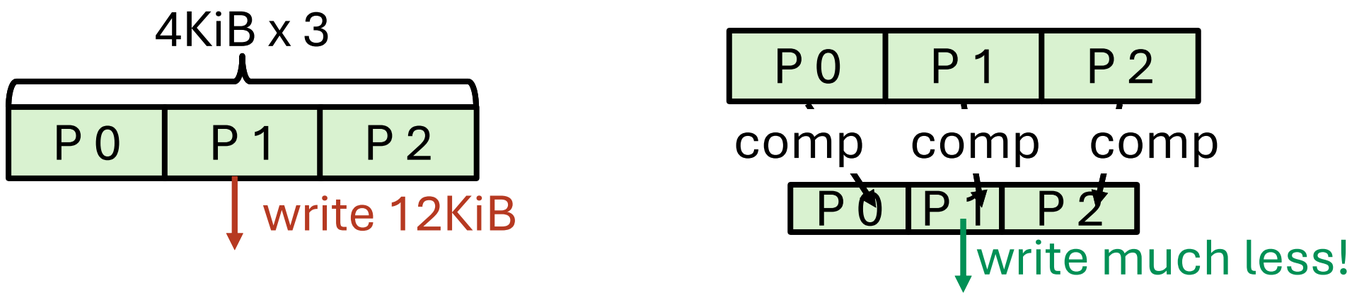
standard SSD
NoWA pattern

FDP
placement h



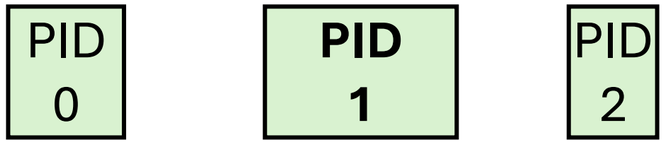
Optimization: page-level compression

- Compression **reduces write and space**
- Out-of-place writes make compression so much **easier**



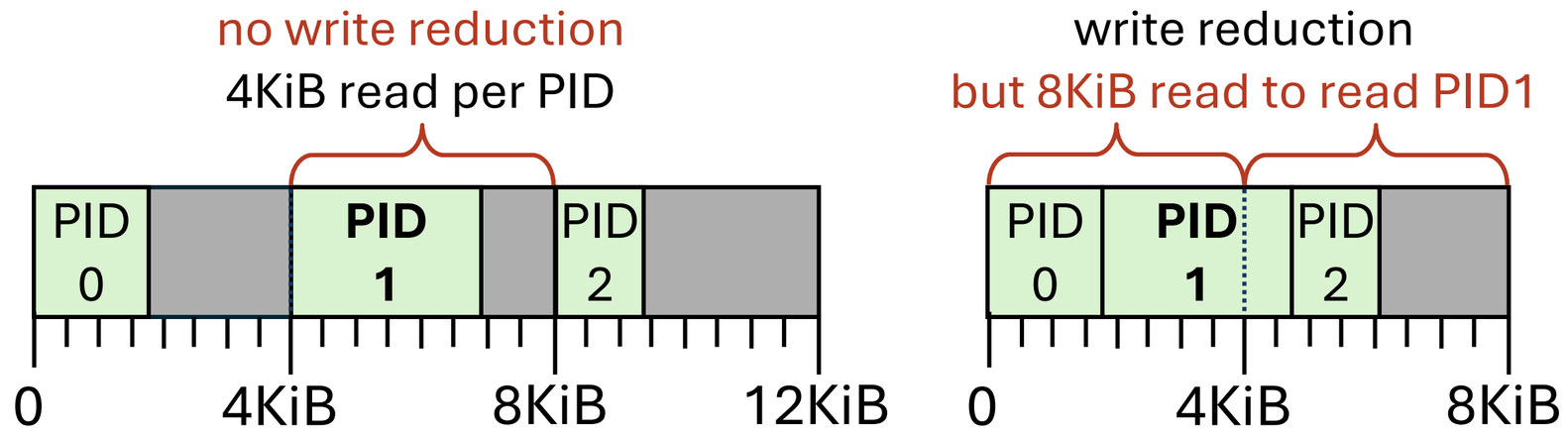
Optimization: 4KiB-granular page packing

- Now we have compressed pages < 4KiB



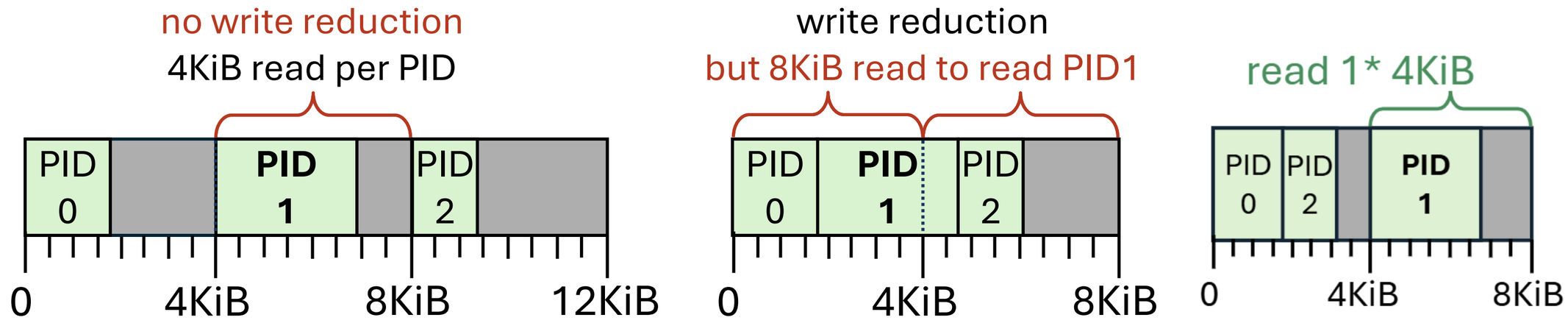
Optimization: 4KiB-granular page packing

- Now we have compressed pages < 4KiB
- Placing compressed pages is tricky due to block alignment restriction



Optimization: 4KiB-granular page packing

- Now we have compressed pages < 4KiB
- Placing compressed pages is tricky due to block alignment restriction
- Pack PIDs so that it always resides in aligned 4KiB offset to always guarantee **4KiB read** when reading a compressed page



Proposed optimizations

in-place write

↓

switch to *out-of-place write*

↓

+ compression and page packing

↓

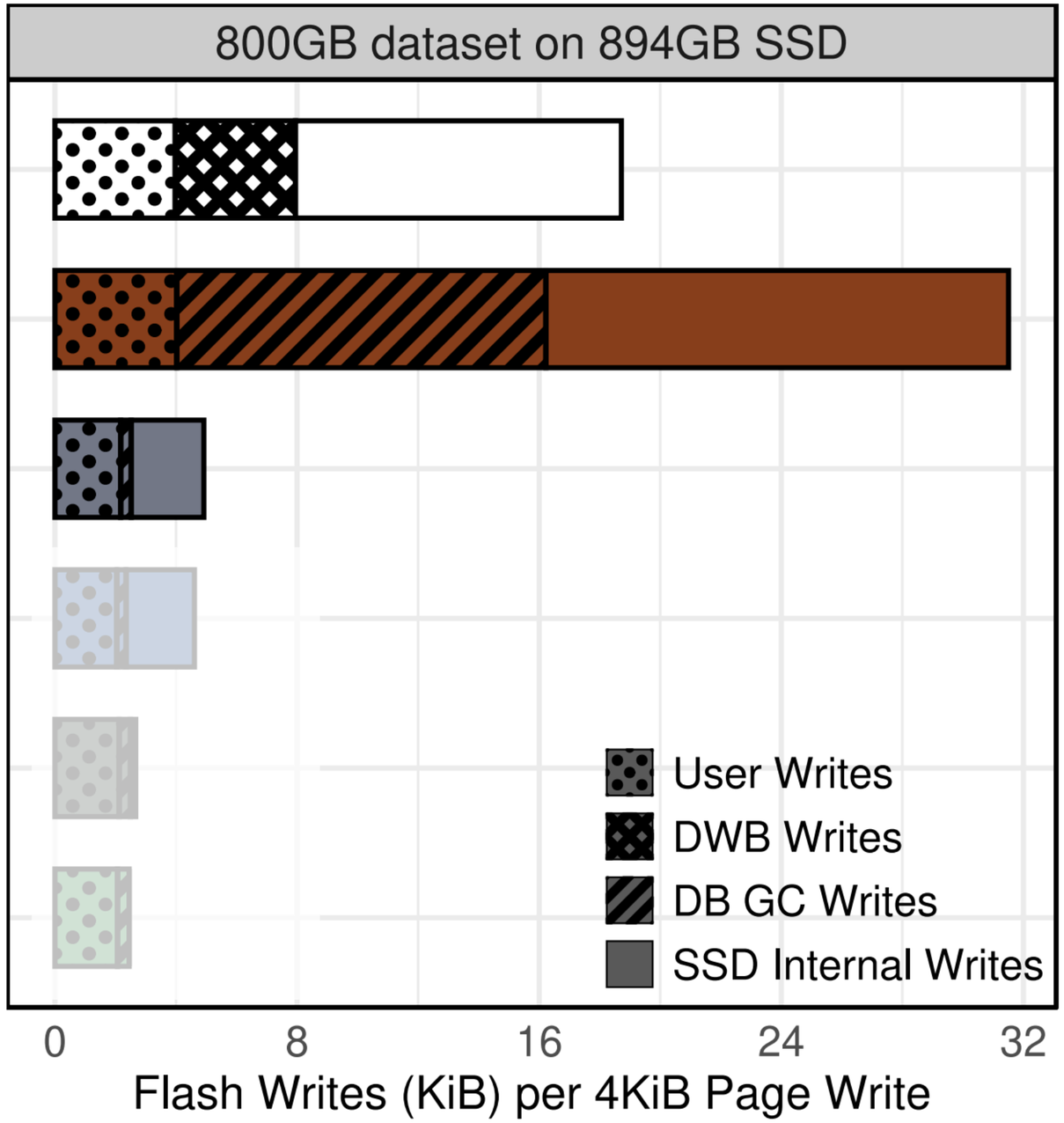
+ deathtime-based placement & GC

↓

+ aligning DB SSD GC unit size

↓

+ *SSD WAF = 1*

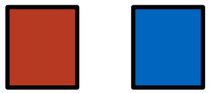


Out-of-place write & Garbage Collection

- Out-of-place write requires GC
- Log-structured filesystem like space management
- GC results in WA while reclaiming invalidated space
- *Victim selection policy* varies WAF

DBMS can do more intelligent GC

- Various related papers exist in filesystem/SSD community



Hot/cold separation

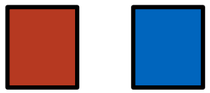
Lifetime-based GC

Deathtime-based GC

- But they are inherently limited as they lack workload knowledge

DBMS can do more intelligent GC

- Various related papers exist in filesystem/SSD community



Hot/cold separation

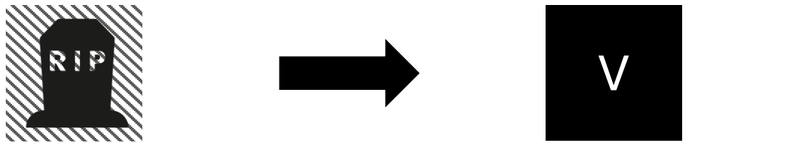
Lifetime-based GC

Deathtime-based GC

- But they are inherently limited as they lack workload knowledge
- But DBMSs can infer the workload better as it sits at the highest application layer

Optimization: Grouping By Deathtime (GDT)

- Previous page versions **“die”** once it is written to a new location
- **Deathtime** = LSN when the next page version is flushed to disk

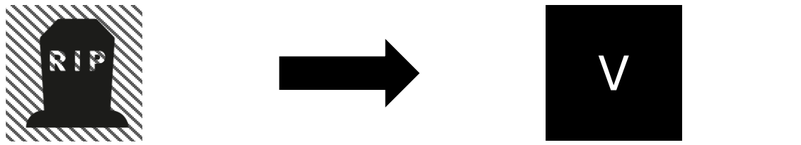


written at LSN 5
died at LSN 10

written at LSN 10

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

written at LSN 10

- **How can we use deathtime?**

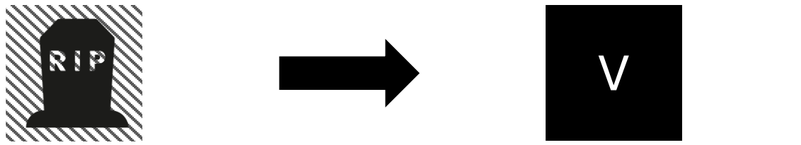
(1) Predict each PID’s next deathtime

(2) Place PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon GC

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

written at LSN 10

- How can we use deathtime?

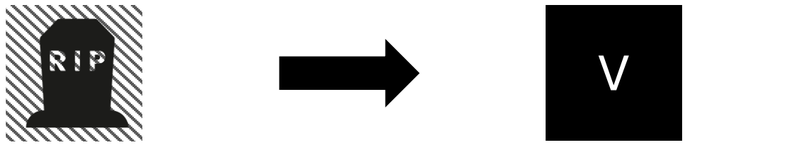
(1) **Predict** each PID’s next deathtime on **write history**

(2) Place PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon GC

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

written at LSN 10

- How can we use deathtime?

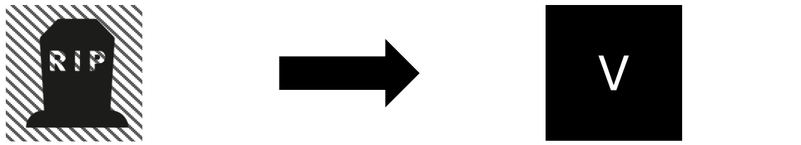
(1) Predict each PID’s next deathtime on write history

(2) **Place** PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon GC

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

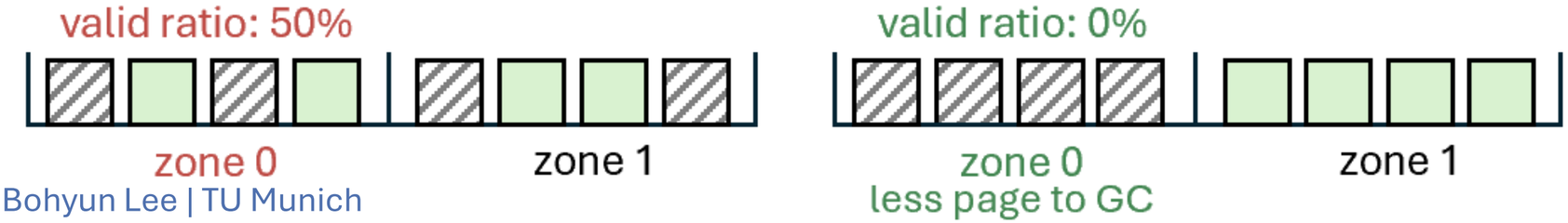
written at LSN 10

- How can we use deathtime?

(1) Predict each PID’s next deathtime based on write history

(2) Place PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon **GC**



Proposed optimizations

in-place write

↓

switch to *out-of-place write*

↓

+ compression and page packing

↓

+ deathtime-based placement & GC

↓

+ aligning DB SSD GC unit size

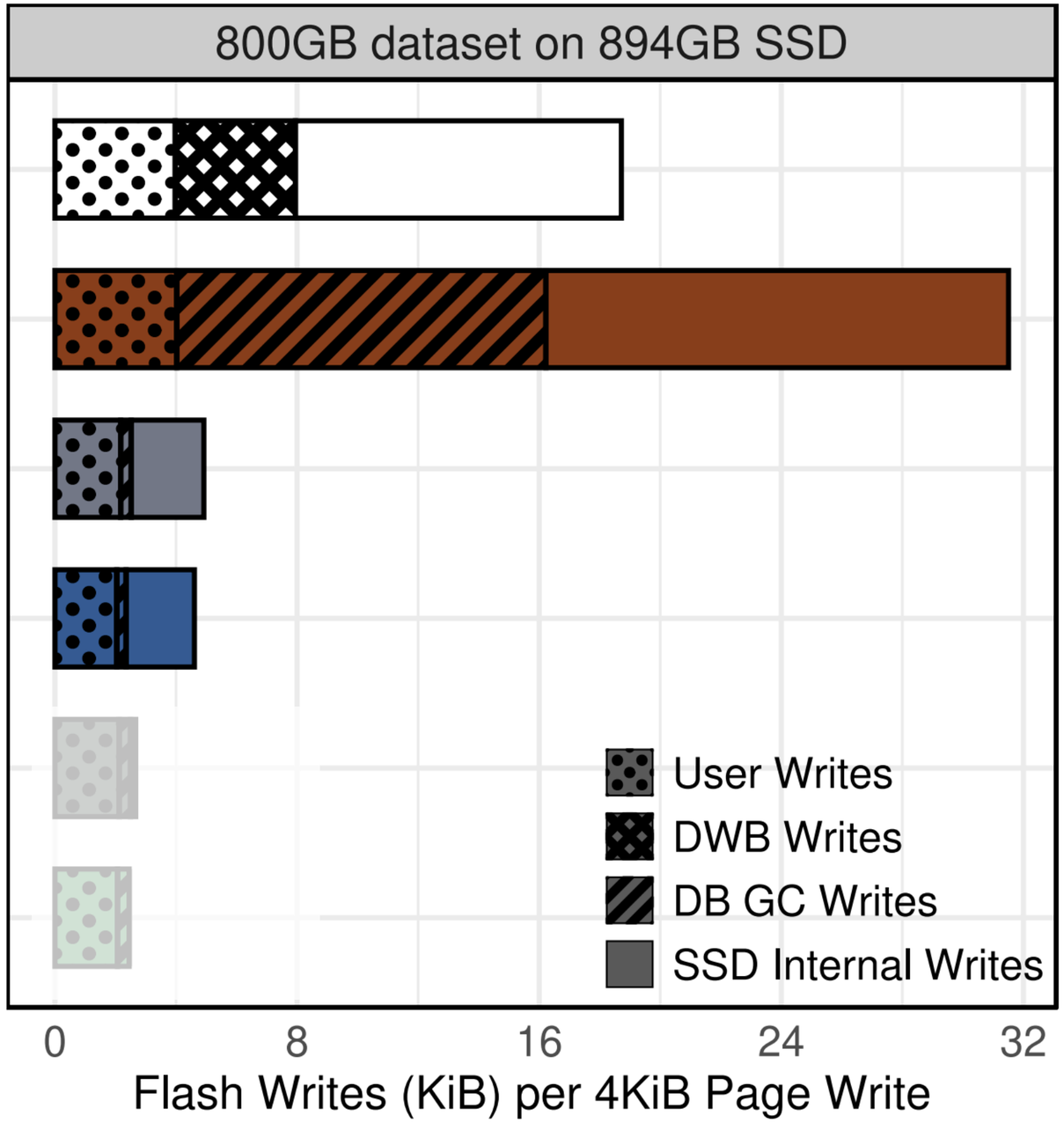
↓

+ *SSD WAF = 1*

ZNS
zone cmd

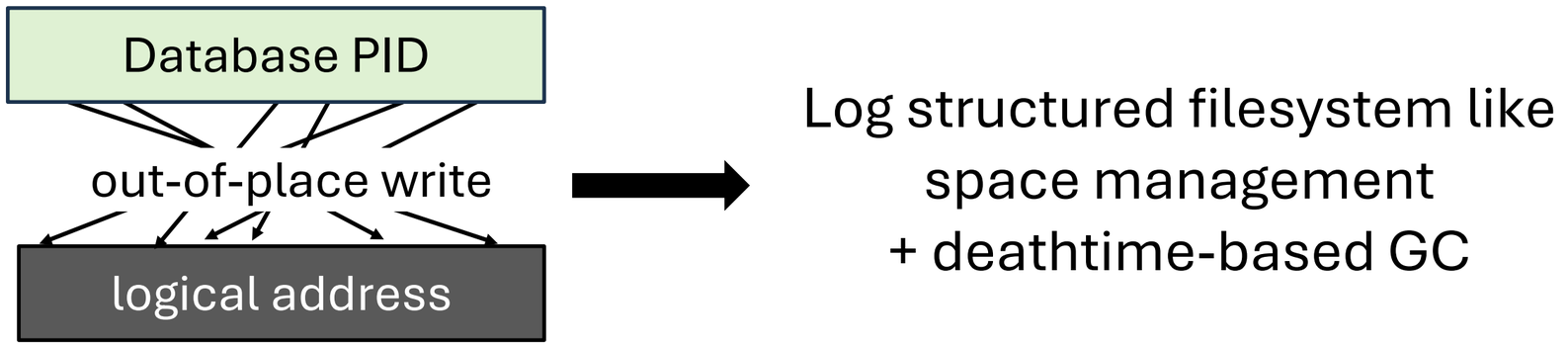
standard SSD
NoWA pattern

FDP
placement h



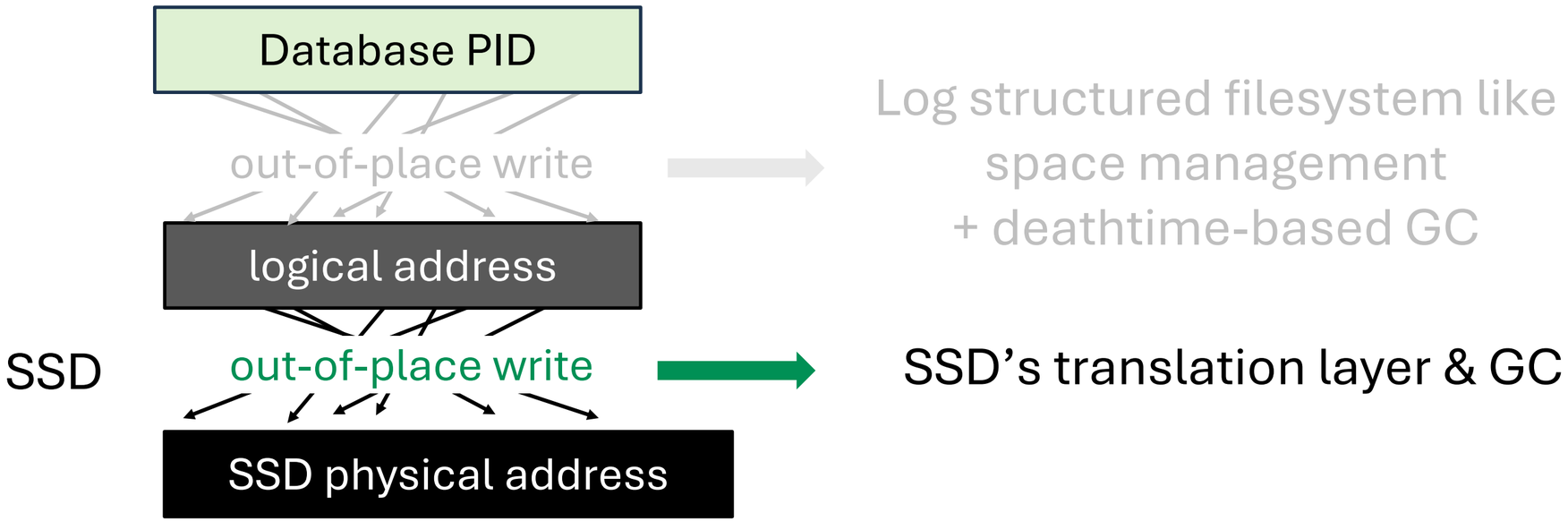
So far...

- We have DBMS doing out-of-place write and GC



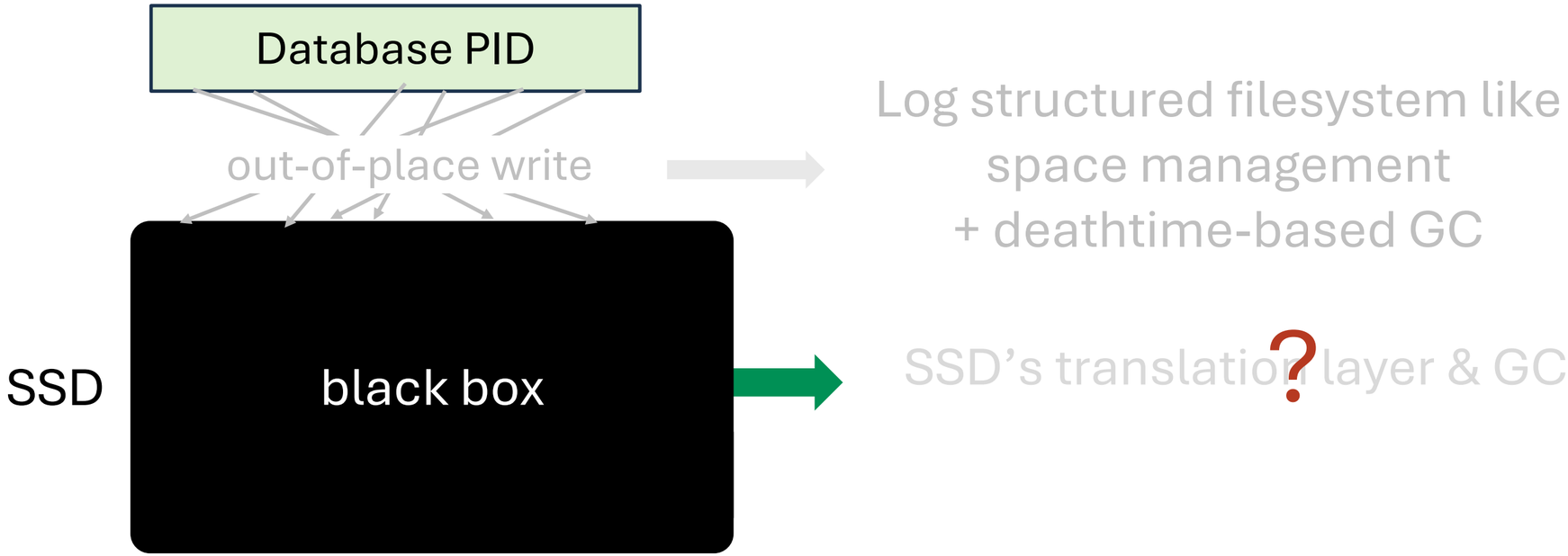
SSDs also do out-of-place write

- Turns out SSDs also do out-of-place write and GC



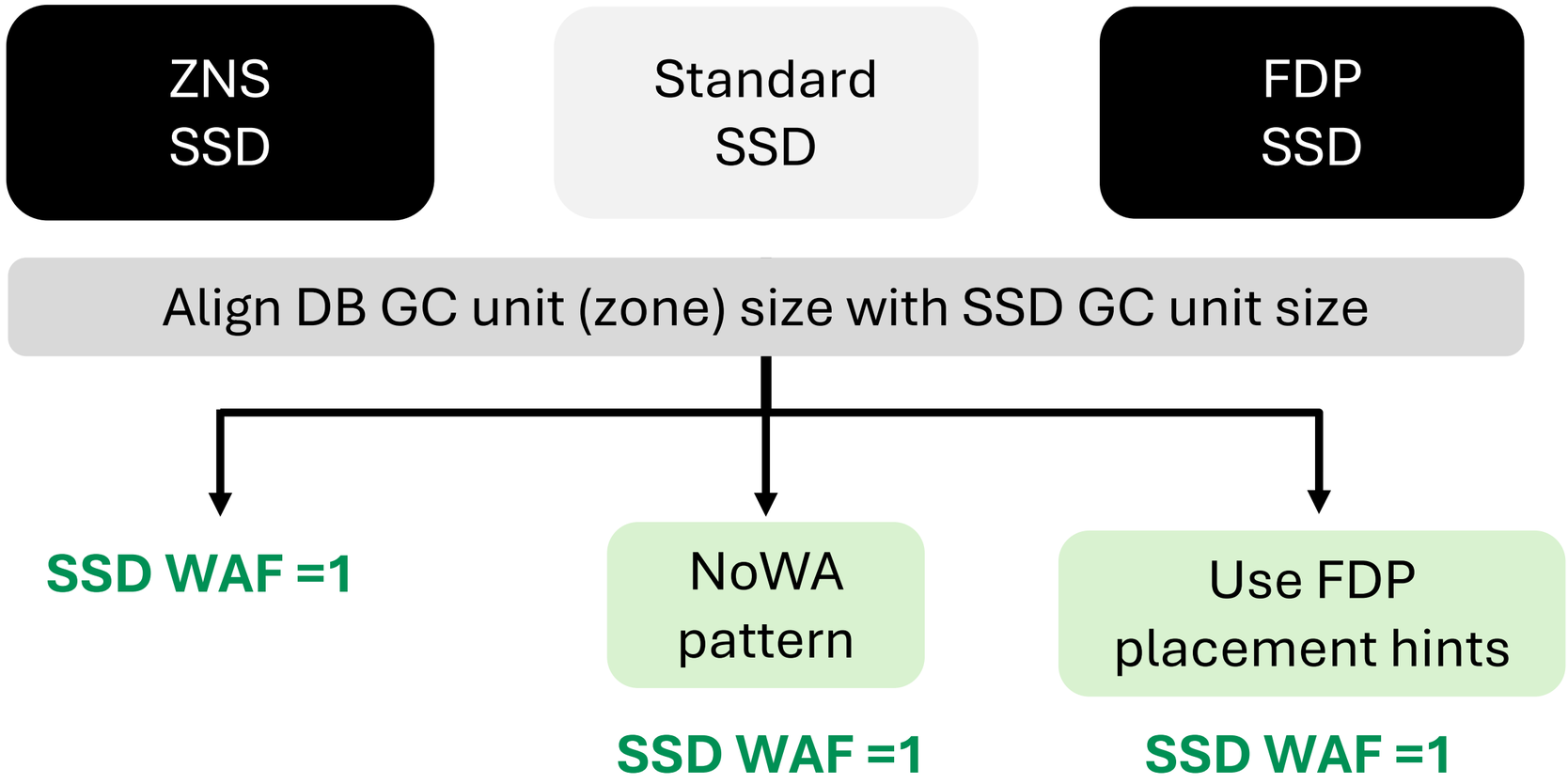
SSDs are blackboxes

- But, we don’t know what’s happening inside the **blackbox**



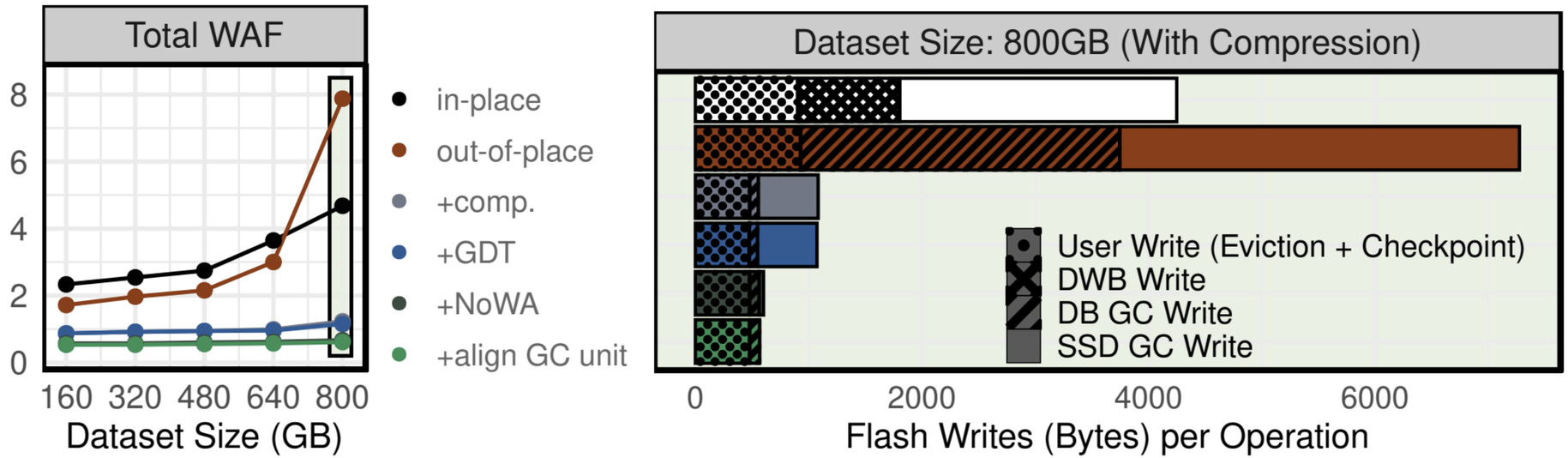
SSD WAF=1 with three different SSD types

- Achieve **SSD WAF =1** even with *full capacity usage*



Evaluation Result

Main Performance Result



(a) Total WAF (b) Write drilldown with compression

Total WAF on different dataset sizes and write drilldown with 800 GB dataset (YCSB-A zipf θ = 0.8, Samsung PM9A3).

Conclusion

Proposed optimizations

switch from in-place write
to *out-of-place write*

↓

\+ per-page compression

↓

\+ page packing

↓

\+ deathtime-based placement & GC

↓

\+ aligning DB SSD GC unit size

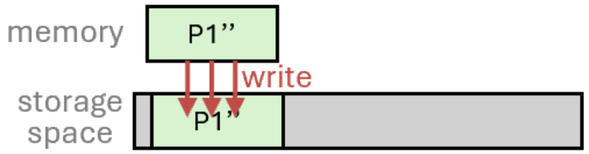
↓

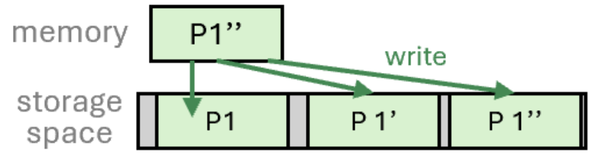
\+ *SSD WAF = 1*

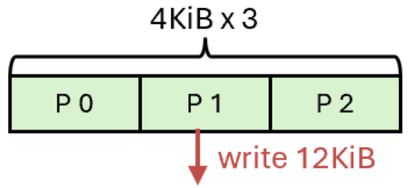
ZNS
zone cmd

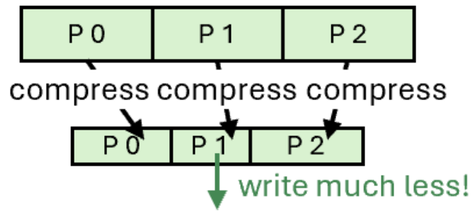
standard SSD
NoWA pattern

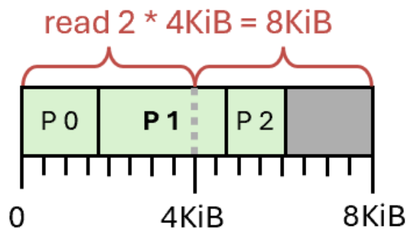
FDP
placement hints

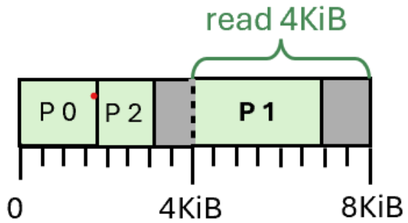


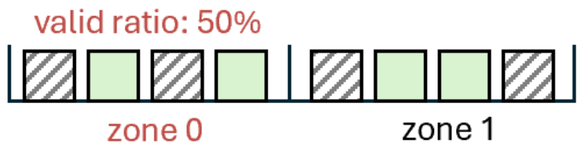


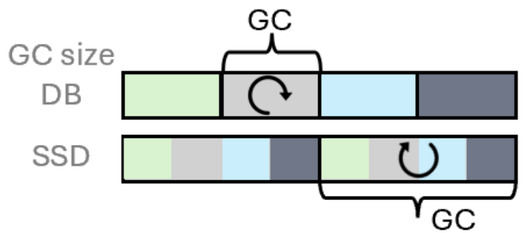


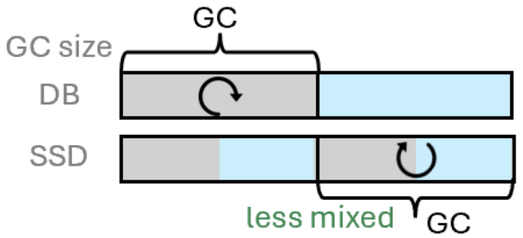


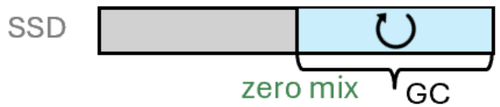












Proposed optimizations

switch from in-place write
to *out-of-place write*

↓

+ per-page compression

↓

It seems like a lot of work.

+ page packing

↓

+ deathtime-based placement & GC

↓

+ aligning DB SSD GC unit size

↓

+ *SSD WAF = 1*

ZNS
zone cmd

standard SSD
NoWA pattern

FDP
placement hints

Proposed optimizations

switch from in-place write
to *out-of-place write*

↓

+ per-page compression

↓

+ page packing

↓

+ deathtime-based placement & GC

↓

+ aligning DB SSD GC unit size

↓

+ *SSD WAF = 1*

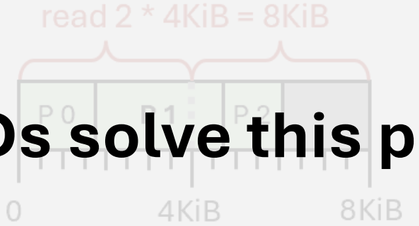
ZNS
zone cmd

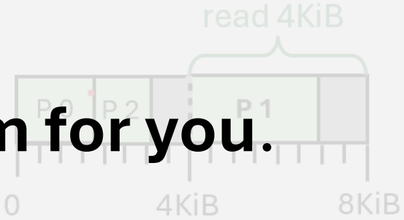
standard SSD
NoWA pattern

FDP
placement hints

It seems like a lot of work.

But neither filesystems nor SSDs solve this problem for you.





Proposed optimizations

switch from in-place write
to *out-of-place write*

↓

\+ per-page compression

↓

\+ page packing

↓

\+ deathtime-based placement & GC

↓

\+ aligning DB SSD GC unit size

↓

\+ *SSD WAF = 1*

ZNS
zone cmd

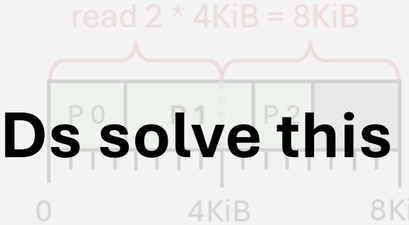
standard SSD
NoWA pattern

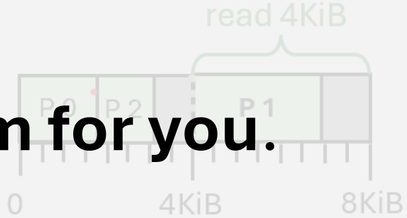
FDP
placement hints

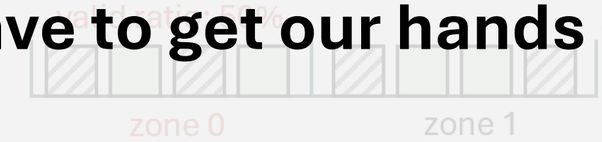
It seems like a lot of work.

But neither filesystems nor SSDs solve this problem for you.

So, us, DBMSes, have to get our hands dirty!



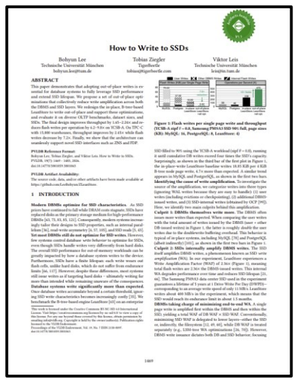




Questions?

extended version

VLDB version



Code available at

Varying Benchmark : TPC-C

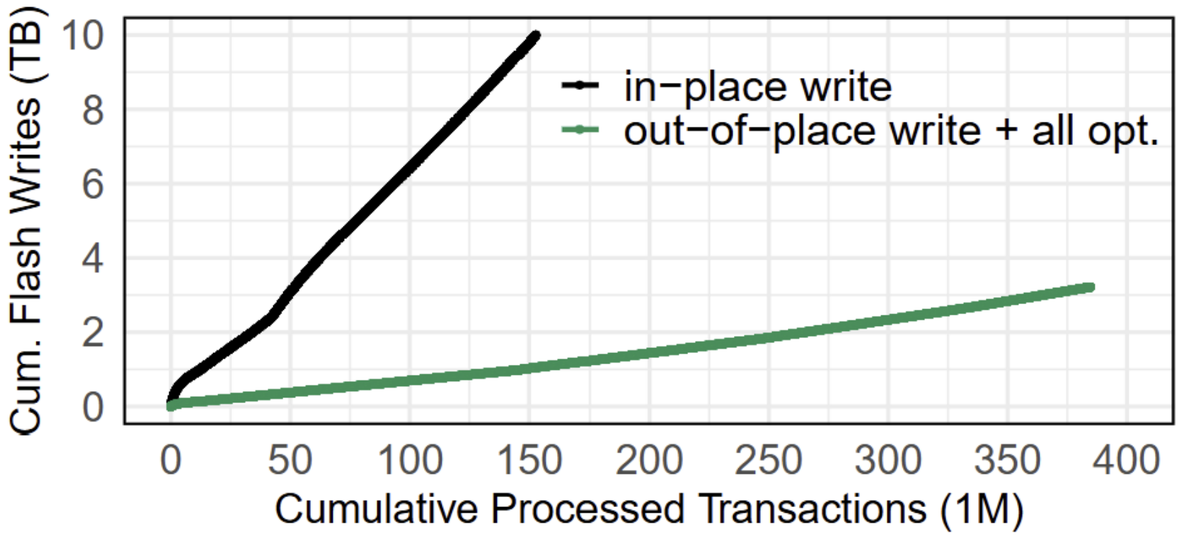


Figure 16: Comparing cumulative flash writes per tranasction: TPC-C (15,000 warehouses (=1,600GB), 32 clients, FDP SSD A)

Varying Skew Factor

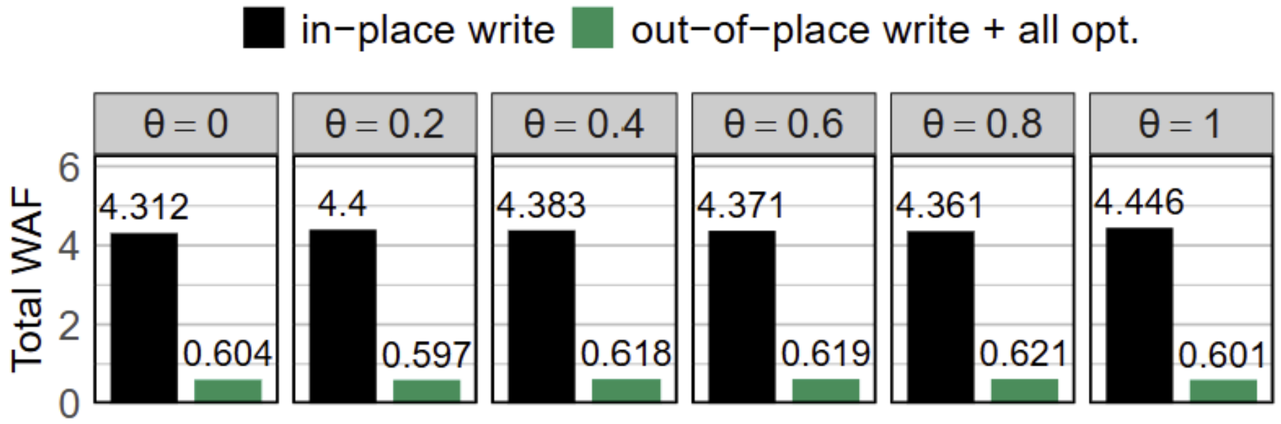


Figure 14: Total WAF while varying skewness. (YCSB-A, DB size: 800GB, Micron 7450 PRO)

Varying SSDs

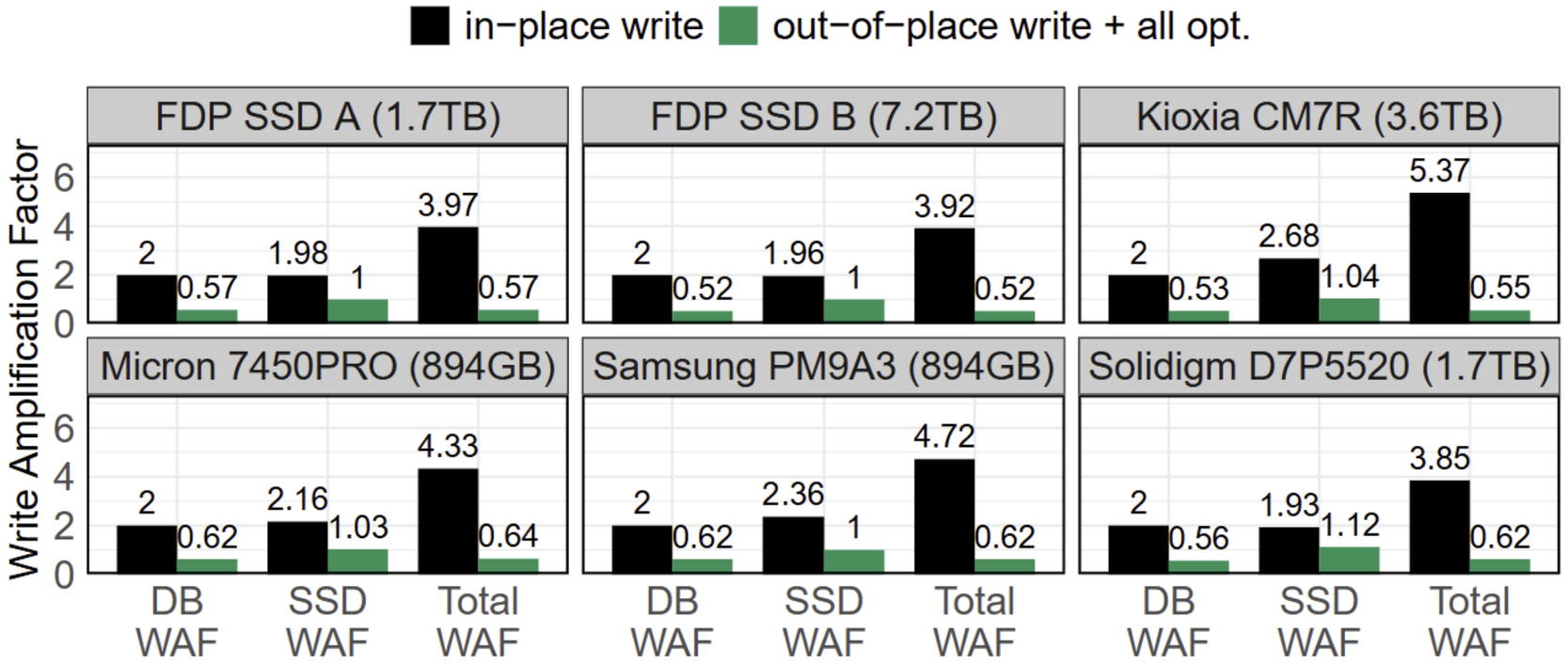


Figure 12: WAF comparison across different SSD models (YCSB-A zipfian θ = 0.8, SSD space 90% full)

Varying Dataset Size

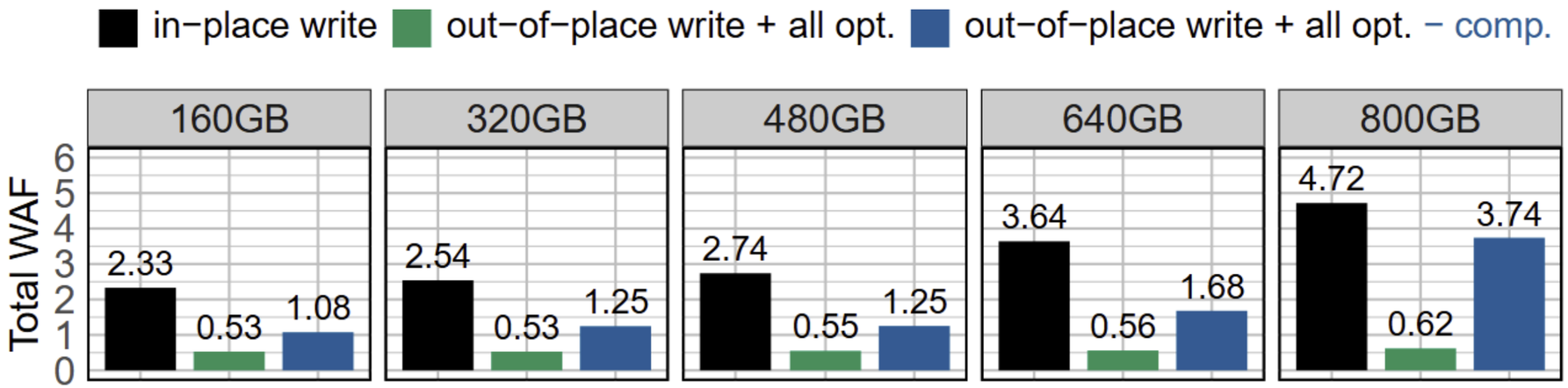
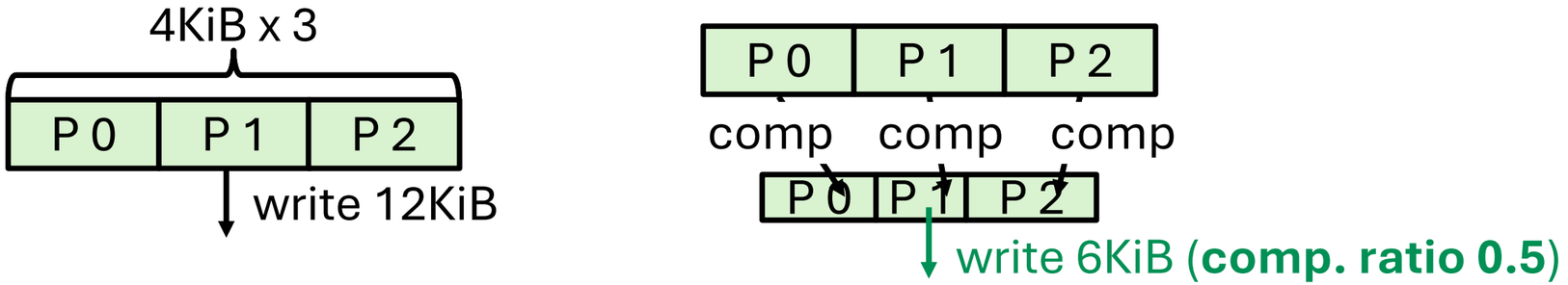


Figure 13: Total WAF while varying dataset size (YCSB-A zipfian θ = 0.8, Samsung PM9A3)

Optimization: page-level compression

- Compression reduces write and space
- Out-of-place writes make compression so much easier



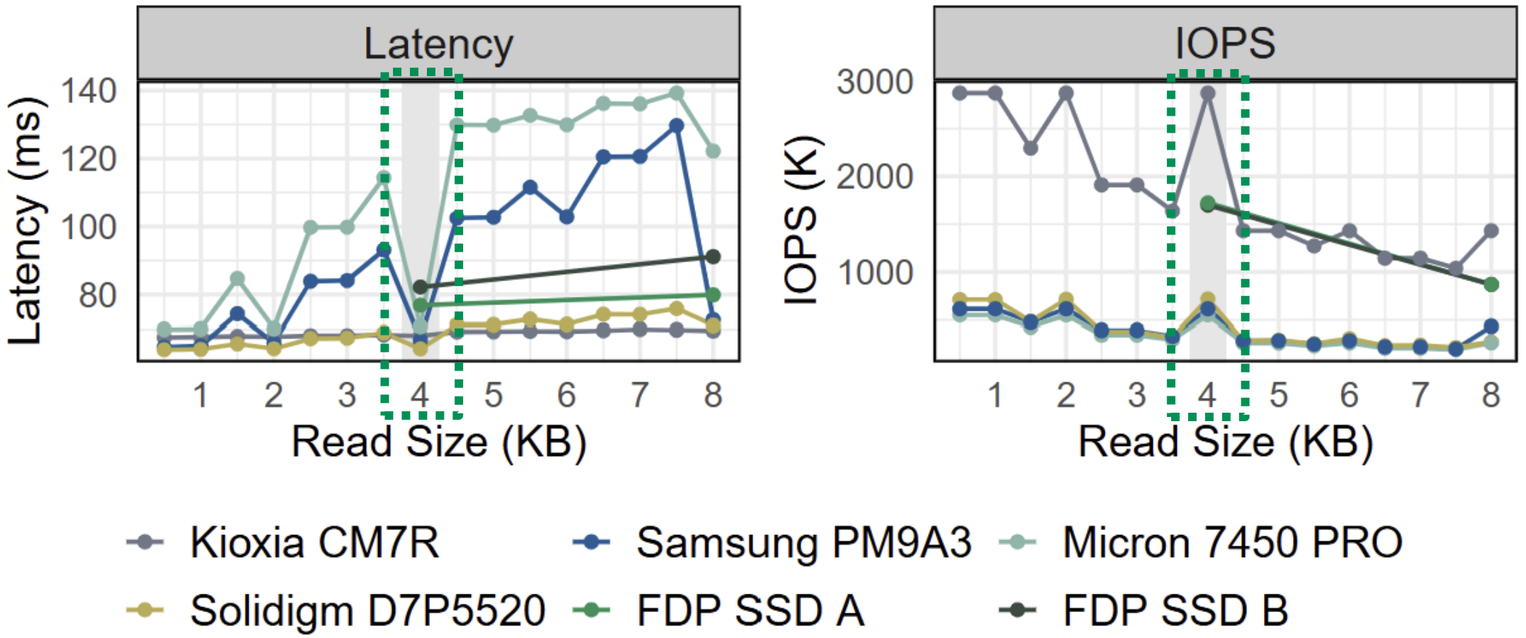
- This reduction depends on the **compression ratio**
- But real-world datasets are oftentimes compressible (e.g., 0.15-0.5)

Compression ratio of various datasets

Comp. Ratio (%)	TPC-C	YCSB	Email	URL	Wikipedia
LZ4 [23]	49.0	41.2	40.1	25.0	31.3
ZSTD [24]	36.5	34.4	27.4	14.5	17.8
zlib [40]	38.1	34.3	25.5	15.5	19.0

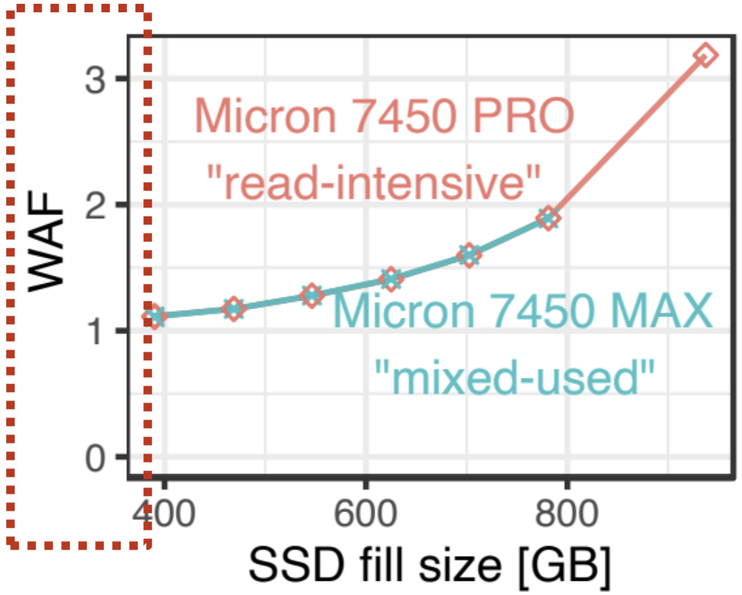
Choosing 4KiB as a page size

- ***4KiB*** gives the best **read latency** and highest **read throughput**



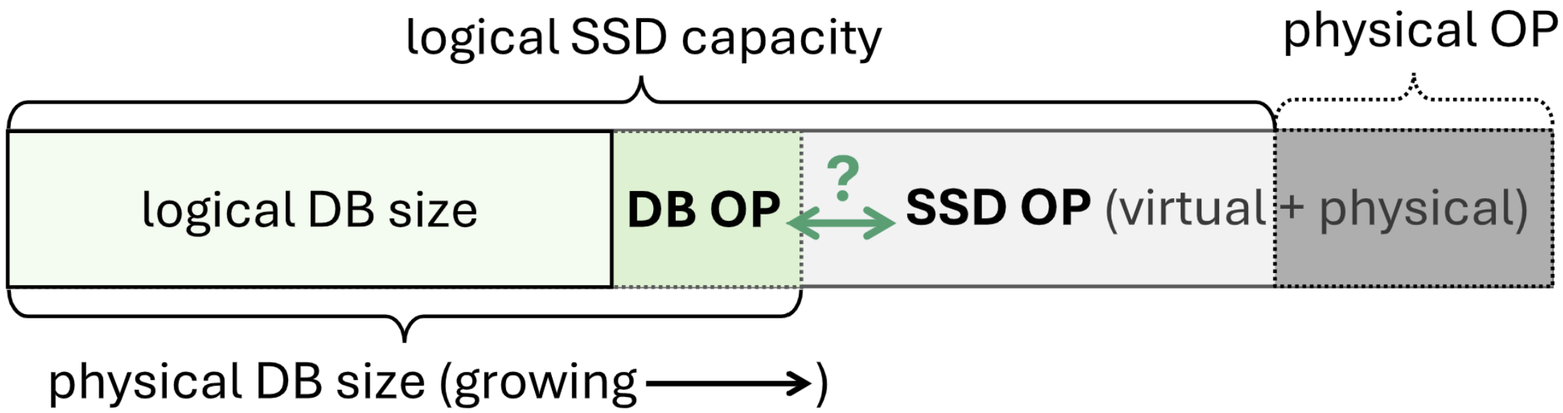
Modern SSDs can exhibit very high WAF

- SSD WAF can be very high, depending on various factors like **fill factor**



SSDiq: Haas et.al. VLDB25

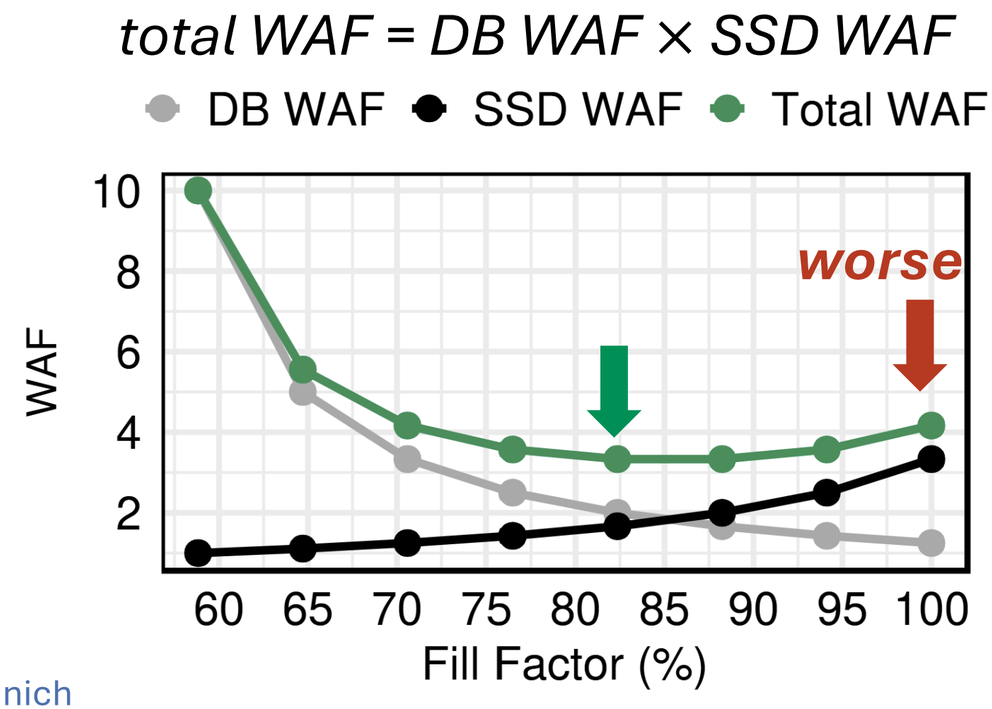
Two layers of indirection and GC



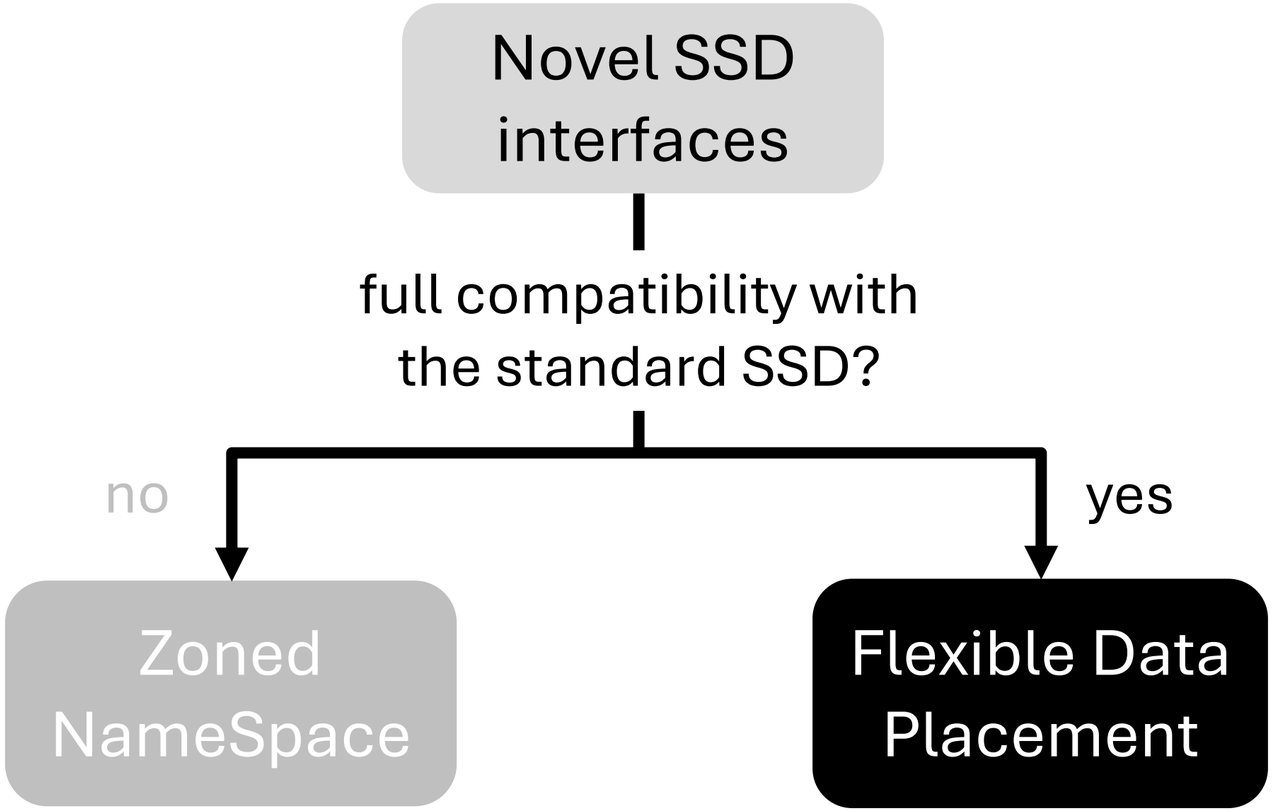
- DB GC is fighting against SSD GC for limited OP space
- DB GC triggering point splits the DB OP and SSD OP space
- Thus, DB GC behavior can shape **both** DB WAF and SSD WAF

DB GC decides both DB and SSD WAF

- Suppose we use full SSD capacity to lower DB WAF
- But in return, it can paradoxically result in more flash writes



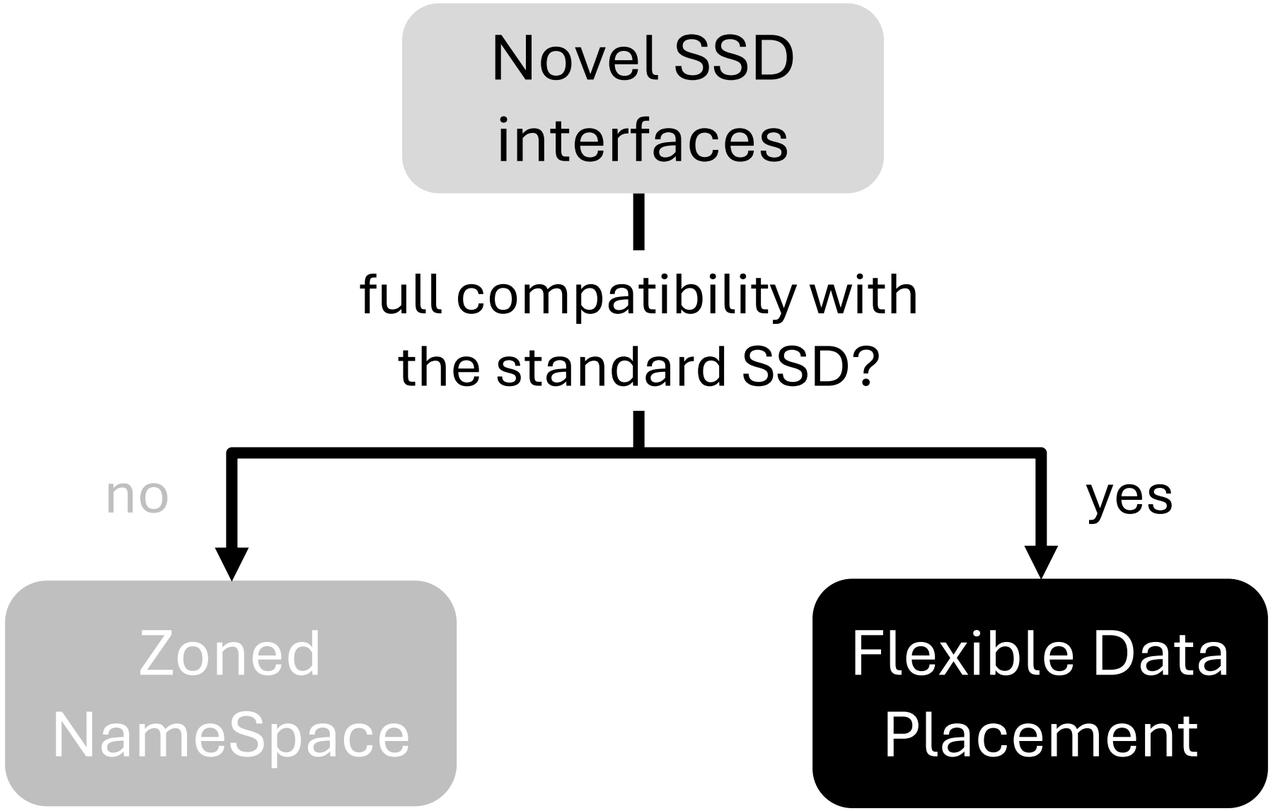
Different SSD interfaces to address WAF



- zone interface (append only)
- no SSD GC, no SSD WAF
- but GC offloaded to host

- block interface
- SSD WAF can persist
- but can be mitigated by placement hints from host

Different SSD interfaces to address WAF



- zone interface (append only)
- no SSD GC, no SSD WAF
- but GC offloaded to host

- block interface
- SSD WAF can persist
- but can be mitigated by placement hints from host

Overview of Write Path